SEA-NET: A SMALL AND INHERENTLY INTERPRETABLE NEURAL NETWORK FOR TIME SERIES CLASSIFICATION

MASTER'S INTERNSHIP REPORT

Ubaid Ur Rehman
M2 DSAI
CRIStAL Lab / Fox Team / Computer Science Department
ubaid-ur.rehman@etu.univ-cotedazur.fr

Supervisor: Tanmoy MONDAL
Internship period: 01-June-2026 – 30-September-2026
Submitted: 15-Aug-2026

ABSTRACT

Deep learning models classify time series accurately, but most of them cannot say *which part* of the signal led to their decision. The usual answer is to explain the model after training with a separate tool, which adds cost and gives an explanation that is not guaranteed to match what the model actually computed. The MILLET framework solves this by building the explanation into the model itself: it scores every time step, and those scores are both the prediction and its explanation. MILLET is accurate and interpretable, but it is large – about 424 K parameters – which rules it out for the microcontrollers that motivate this internship. During this internship I designed SEA-Net, a compact and inherently interpretable architecture for time series classification. It has two parts: a family of small multi-scale depthwise-separable convolutional *encoders*, and a family of seven new *Multiple Instance Learning (MIL) pooling heads*, each one designed to remove a specific limitation of the pooling method used by MILLET.

I evaluated 72 encoder and pooling-head combinations on the WebTraffic dataset and on the UCR archive, measuring classification quality (accuracy) and explanation quality (AOPCR and NDCG@n), with three random seeds on the models that carry the main comparison. On WebTraffic, SEA-Net bottleneck encoder + Top-k pooling reaches 0.947 accuracy and 0.773 NDCG@n with 41 K parameters, against 0.887 accuracy and 0.677 NDCG@n for the MILLET baseline re-trained under the same configuration with 424 K parameters: better classification, better explanations and ten times fewer parameters at once. SEA-Net gated encoder + Top-k pooling is the most accurate model tested (0.955 accuracy, 0.750 NDCG@n, 62 K parameters). Over the 85 UCR datasets, however, the MILLET baseline remains ahead on average, a limitation I report and analyse rather than hide. The conclusion is that inherent interpretability and a small parameter budget are compatible, and that the pooling head is as important as the encoder in reaching both. The code that produced every number in this report is available at https://github.com/ubaidur404786/sea-net.

CONTENTS

1 INTRODUCTION

1.1 PROBLEM STATEMENT

A time series is a sequence of measurements taken in time order, and *time series classification* (TSC) is the task of assigning one label to a complete sequence: normal or abnormal for a heartbeat recording, healthy or faulty for a machine sensor, one traffic pattern or another for a web server log. Deep learning models solve this task well, and convolutional architectures in particular are its standard baselines. Accuracy alone, however, is not enough in the applications that motivate this work, which impose three requirements *at the same time*. The first is **accurate classification**: the model must remain competitive with the strong convolutional baselines used in the TSC literature. The second is **interpretable predictions**: the model must indicate *which time steps* caused its decision, and that indication must be the quantity the decision was actually computed from, not a reconstruction produced afterwards by a separate tool. The third is **a small parameter budget**: the model must be small enough to be a realistic candidate for deployment on an edge device or a microcontroller unit (MCU), where memory is measured in hundreds of kilobytes.

Each requirement is well studied on its own. The difficulty, and the subject of this internship, is that satisfying all three together normally means trading one against the others: accurate models tend to be large, and small models tend to be black boxes.

1.2 MOTIVATION

Time series are produced continuously by sensors close to where the data is generated: medical monitors, industrial equipment, network probes. Sending every measurement to a server is often impossible or undesirable because of bandwidth, latency, energy or privacy, so the classifier has to run on the device itself. The MLPerf Tiny benchmark (Banbury et al., 2021), which defines reference workloads for this setting, uses models of roughly 38.6 K parameters – a useful order of magnitude for what "small enough" means here.

Interpretability matters in the same applications for a practical reason. When a model flags an anomaly in a patient recording or in a machine's vibration signal, the person who must act on that alert needs to know which part of the signal triggered it, both to trust the alert and to decide what to do about it. Rudin (2019) argues that in high-stakes settings a model that is interpretable *by design* should be preferred to a black box paired with an after-the-fact explanation, because only the first guarantees that the explanation and the decision are the same computation.

1.3 STATE OF THE ART

Deep learning for time series classification. Wang et al. (2017) showed that two simple convolutional networks – a Fully Convolutional Network (FCN) and a residual network (ResNet) – are strong baselines for TSC and they remain reference points today. InceptionTime (Ismail Fawaz et al., 2020) extends this line by applying filters of several kernel sizes in parallel inside one module, capturing patterns of different durations without choosing a scale in advance; it is the most accurate model of this family and the backbone the baseline of this work builds on. Temporal Convolutional Networks (Bai et al., 2018) contribute the second ingredient reused here, *dilated* convolutions, which widen the region a filter sees without adding weights. Ismail Fawaz et al. (2019) survey the family and confirm that these models dominate on the UCR archive (Dau et al., 2019). All of them share one property that matters here: they end with global average pooling, which collapses the time axis into a single vector and destroys the per-time-step information an explanation would need.

Post-hoc explanation methods. Because of that limitation, such a model is usually explained by a method applied after training, on the frozen model. LIME (Ribeiro et al., 2016) fits a simple readable model, for example a sparse linear one, to the black box's behaviour in a small neighbourhood around one input, and reads each feature's importance off that surrogate. SHAP (Lundberg & Lee, 2017) assigns each feature a contribution derived from Shapley values in cooperative game theory, giving the attribution a set of formal properties. Saliency methods, of which Integrated Gradients (Sundararajan et al., 2017) is a widely used representative, use the gradient of the output with respect to the input to score how sensitive the prediction is to each position. All three are *post-hoc*: separate computations on top of a trained model, adding inference cost, and returning an approximation of the model that can disagree with what the model actually computed. That last point is the argument of Rudin (2019) and the reason this work follows the interpretable-by-design route.

Interpretable-by-design models for time series. Two directions exist. The first goes through *shapelets*, short representative subsequences whose presence explains the class: VQShape (Wen et al., 2024) learns a codebook of abstracted shapes and represents a series as tokens drawn from it, so a prediction reads as a set of recognised shapes, while InterpGN (Wen et al., 2025) combines a shapelet expert with a deep expert through a confidence gate. The

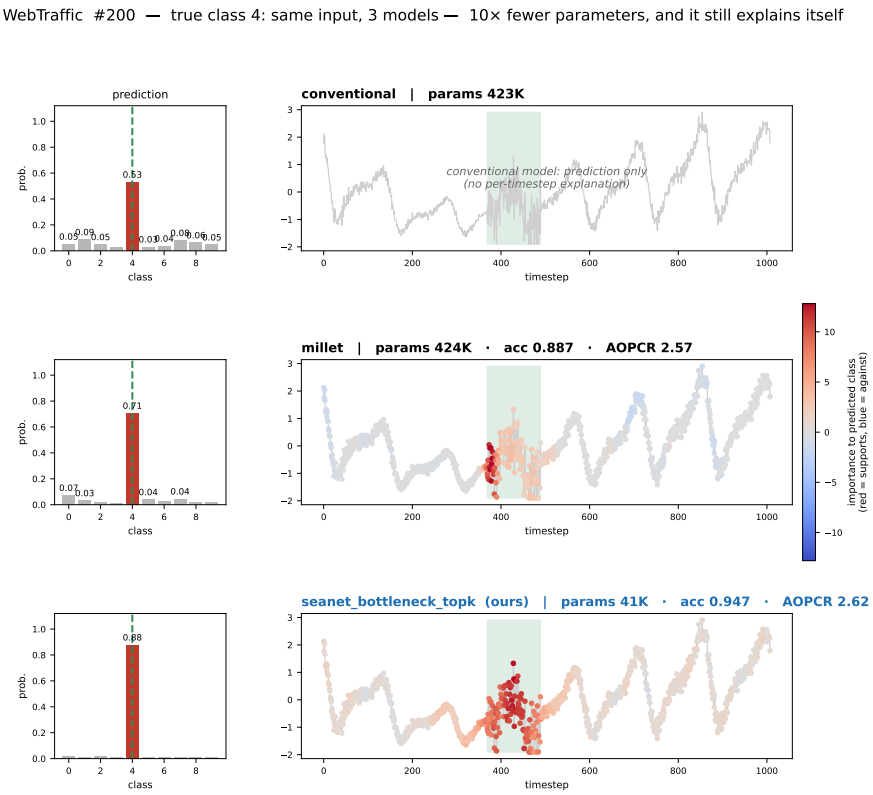


Figure 1: The same WebTraffic series classified by three models. Each panel shows the predicted class and, below it, the explanation the model can give for that prediction; red marks the evidence supporting the predicted class at each time step. The "true evidence" row is the generator's own record of the time steps it modified and was never used during training. A standard black-box convolutional model (left) predicts class 4 with 53 % confidence and offers no explanation. MILLET (middle) predicts the same class with 71 % confidence and does explain itself, but needs 424 K parameters. SEA-Net (right) is more confident, produces a sharper explanation, and uses 41 K parameters.

second direction, the one this report follows, uses Multiple Instance Learning (MIL). MILLET (Early et al., 2024) treats a series as a bag of time steps and replaces global average pooling with a MIL pooling head that scores every time step; the class prediction is computed from those scores, so they are simultaneously the prediction mechanism and the explanation and no second computation is needed. MILLET is the baseline of this work and is described in Section 2.3. Its pooling head descends from attention-based MIL (Ilse et al., 2018), developed for histopathology images – a literature this report also borrows from (Section 2.2).

Small models for edge deployment. Depthwise-separable convolutions (Howard et al., 2017; Chollet, 2017) reduce the cost of a convolution by splitting it into a per-channel filtering step and a channel-mixing step, and are the standard tool for building small convolutional networks. MLPerf Tiny (Banbury et al., 2021) provides reference models and tasks for microcontroller-class hardware, and post-training quantisation (Jacob et al., 2018) shrinks a trained model further by storing and computing its weights in low-precision integers. These works make small models accurate, but none of them makes them interpretable.

The gap. Putting the two lines side by side gives the problem this internship addresses. MILLET provides classification *and* a faithful built-in explanation, but its InceptionTime backbone needs about 424 K parameters; the edge-oriented models are small enough for a microcontroller but output a class label only. No existing model delivers all three requirements of Section 1.1 at once.

1.4 Contributions

To close that gap I designed SEA-Net, an inherently interpretable architecture for time series classification proposed in this report and not a pre-existing model. The name stands for *Selective Evidence Aggregation Network*, because it combines multi-scale temporal feature extraction with selective, class-specific aggregation of the evidence found at each time step. It has two independently replaceable parts – a SEA-Net *encoder* and a MIL *pooling head* – and my contributions cover both, plus the infrastructure needed to compare them:

My contribution.

- **The SEA-Net encoder and its variants.** I designed a compact encoder built from multi-scale depthwise-separable dilated convolutional blocks, and seven SEA-Net encoder variants on top of it: bottleneck, gated, input-gated, spike/trend, reconstruction-residual, and two wrappers (multi-scale channels and multi-scale pyramid). Every variant preserves the one embedding per time step that an inherent explanation requires (Section 3.1).
- **Seven new MIL pooling heads.** I designed and implemented seven MIL pooling variants – class-wise conjunctive, softmax conjunctive with a learnable sharpness parameter, adaptive class-wise, Top-k conjunctive, attention-max, per-class gated attention and dual-stream conjunctive. Each one targets a specific limitation of the pooling head used by MILLET, and five of them provably reduce to that baseline for one setting of their parameters (Section 3.4).
- **A modular, configuration-driven benchmark.** I built a pipeline in which any encoder can be paired with any pooling head from a single configuration file, with resumable multi-dataset runs, experiment tracking, and automatic regeneration of every figure and table in this report from the stored results (Section 4.2).
- **An empirical study of 72 encoder and pooling-head combinations** over WebTraffic and the UCR archive, measuring accuracy, explanation quality and model cost together, against FCN, ResNet and MILLET baselines that I re-trained myself under an identical training configuration (Section 5).

Outline. Section 2 gives the background: notation, Multiple Instance Learning, the MILLET baseline and its limitations, and the two explanation-quality metrics. Section 3 presents the proposed encoders, pooling heads and training objective; Section 4 the experimental methodology; Section 5 the results and Section 6 their analysis. Section 7 concludes and lists future work. Appendix A covers the internship itself – host team, objectives, constraints and skills – and Appendix B the supporting results. The material that supports those results without being needed to follow them is published with the code (Section 4.2).

2 Background

2.1 Time series classification and notation

A univariate time series of length T is an ordered list of measurements $\mathcal{X} = (x_1, \dots, x_T)$, and time series classification assigns to that complete list a single label $y \in \{1, \dots, C\}$. Because the value at one time step depends mostly on its neighbours, convolutional models fit this data well, which is why the standard deep baselines (FCN, ResNet and InceptionTime, Section 1.3) are all convolutional. All three end with global average pooling, which averages the T time steps into a single vector before the classifier and therefore removes the per-time-step information an explanation requires. Building an encoder that never removes that information is the object of Section 3.1.

Table 1 defines the symbols used in the rest of this report. They follow Early et al. (2024) so that the proposed pooling heads and the baseline can be compared line by line. These symbols are not redefined later.

Table 1: Notation used throughout this report.

Symbol	Meaning	Symbol	Meaning
$\mathcal{X} = (x_1, \dots, x_T)$	input series (the bag)	$\mathbf{z}_j \in \mathbb{R}^d$	embedding of time step j
x_j	value at time step j (an instance)	$\mathbf{p}_j \in \mathbb{R}^C$	per-time-step class logits
T	number of time steps	a_j, a_j^k	attention gate (shared / per class)
C	number of classes	$g_j^k = a_j^k \mathbf{p}_j^k$	evidence of step j for class k
y	class label of the series	$\hat{\mathbf{Y}} \in \mathbb{R}^C$	bag logits (the prediction)
B	batch size	$\Phi \in \mathbb{R}^{C \times T}$	interpretation (importance map)
d	encoder width (channels)	$\mathbf{h} \in \mathbb{R}^{d \times T}$	feature map inside the encoder
d_a	attention hidden width	$\odot$	element-wise product
δ	dilation of a convolution	$[\cdot\,;\,\cdot]$	concatenation along channels
$\mathcal{K}$	set of kernel sizes in a block	σ	sigmoid function

2.2 Multiple Instance Learning

Bags, instances, and weak supervision. Multiple Instance Learning (Dietterich et al., 1997) is a form of weakly supervised learning in which training labels are attached to *groups* of examples rather than to individual ones. A group is a *bag* and its members are *instances*; the bag carries a label, the instances do not. In the original formulation of Dietterich et al. (1997) a molecule is a bag and each of its possible three-dimensional shapes is an instance: the molecule is known to bind or not to bind a receptor, but which shape is responsible is unknown. Formally, a bag

$\mathcal{X} = (x_1, \ldots, x_T)$ has an observed label y while the instance labels $y_1, \ldots, y_T$ exist conceptually but are never observed. The problem is to predict y from the bag and, ideally, to recover which instances were responsible.

Why a time series is a bag. This is exactly the structure of time series classification. The series as a whole is labelled, but in most datasets only a few time steps carry the evidence for that label: the abnormal beat in a long recording, the spike in a traffic trace. Treating the series as the bag and each time step as one instance turns the interpretability question into a MIL question – recovering instance-level contributions from a bag-level label. Figure 2 shows the correspondence.

Pooling: how instances become a bag prediction. An encoder f_θ maps every instance to an embedding,

$$\mathbf{z}_j = f_\theta(\mathcal{X})_j \in \mathbb{R}^d, \qquad j = 1, \ldots, T, \tag{1}$$

and a *MIL pooling function* combines the T embeddings into one bag prediction $\hat{\mathbf{Y}} \in \mathbb{R}^C$. The choice of pooling function distinguishes MIL methods from one another and determines whether an interpretation can be recovered at all. The two elementary choices are rigid: mean pooling, $\hat{\mathbf{Y}} = \frac{1}{T}\sum_j W_c \mathbf{z}_j$, treats every instance as equally important and cannot single out evidence, while max pooling keeps only the strongest instance and cannot represent evidence spread over several.

Attention-based MIL (Ilse et al., 2018) removes this rigidity by *learning* how much each instance counts. A small network scores each embedding and the scores are normalised over the bag,

$$\alpha_j = \frac{\exp\big(\mathbf{w}^\top \tanh(W\mathbf{z}_j)\big)}{\sum_{i=1}^{T} \exp\big(\mathbf{w}^\top \tanh(W\mathbf{z}_i)\big)}, \qquad \hat{\mathbf{Y}} = W_c \sum_{j=1}^{T} \alpha_j \mathbf{z}_j, \tag{2}$$

so the bag embedding is a weighted average whose weights are produced by the model itself. Because α_j is exactly the quantity used to build the prediction, it can be read directly as an importance score for instance j; no separate explanation step is involved. Ilse et al. (2018) also propose a *gated* version in which a second branch modulates the first, $\tanh(V\mathbf{z}_j) \odot \sigma(U\mathbf{z}_j)$, making the scoring more selective. This is the property that MILLET and every pooling head in this report reuse.

MIL beyond time series. The most active application of MIL is histopathology, where a whole-slide image of 40,000 × 40,000 pixels is cut into thousands of patches: the slide is the bag, each patch an instance, and only the slide-level diagnosis is known. Two methods from that literature are adapted here. MS-DA-MIL (Hashimoto et al., 2020) runs the same attention-based MIL pipeline at several magnifications and fuses the results, on the grounds that evidence appears at different scales. DSMIL (Li et al., 2021) uses two streams: one aggregates all instances, the other identifies the single most critical instance and re-weights the rest by their similarity to it, letting one decisive instance dominate without discarding the others as max pooling would.

2.3 MILLET: THE BASELINE OF THIS WORK

Existing work (reused). MILLET (Early et al., 2024) is the main baseline of this internship. Everything in this subsection is their work. I reuse their model formulation, their metric implementation and their data loaders without modification; my only change was to wrap their code inside the pipeline of Section 4.2 so that their models and mine run under identical conditions. The encoders of Section 3.1 and the pooling heads of Section 3.4 are my own work and are written in the notation used here, so the two can be compared directly.

MILLET applies the MIL view of Section 2.2 to time series classification. An InceptionTime encoder produces one embedding per time step as in Equation (1), and a MIL pooling head turns those embeddings into a bag prediction $\hat{\mathbf{Y}}$ together with an interpretation $\Phi \in \mathbb{R}^{C \times T}$, where $\Phi_{k,j}$ quantifies how much time step j pushed the model towards class k. Their best head, *conjunctive pooling*, combines a single attention gate with a per-time-step classifier:

$$a_j = \sigma\big(\mathbf{w}_a^\top \tanh(W_a \mathbf{z}_j)\big) \in [0, 1], \tag{3}$$

$$\mathbf{p}_j = W_c \mathbf{z}_j + \mathbf{b}_c \in \mathbb{R}^C, \tag{4}$$

$$\hat{\mathbf{Y}}^{\,k} = \frac{1}{T}\sum_{j=1}^{T} a_j \, \mathbf{p}_j^k, \qquad \Phi_{k,j} = a_j \, \mathbf{p}_j^k. \tag{5}$$

The gate a_j decides how much weight time step j receives and the classifier $\mathbf{p}_j$ decides what it argues for, and Equation (5) builds the class score as the average over time of the vote weighted by the attention. The essential property is that $\Phi_{k,j}$ consists of the same terms that are summed to produce $\hat{\mathbf{Y}}^{\,k}$: the explanation cannot disagree with the prediction, because it is the prediction, decomposed.

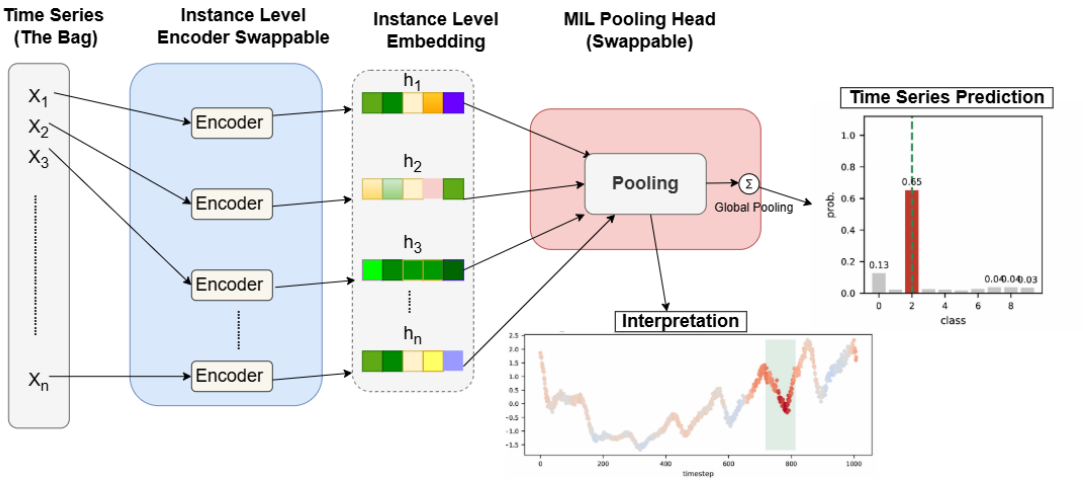


Figure 2: The MIL view of a time series, read left to right. The series is the *bag* and each time step x_j is one *instance*. The same encoder is applied at every time step and produces one instance embedding per step; the MIL pooling head weights those embeddings and combines them into a class prediction (right). Keeping the same weighted contributions instead of discarding them yields the interpretation below: one importance value per time step, drawn on the original series (red = pushes towards the predicted class, blue = pushes away). Both grey boxes are interchangeable, which is what allows any encoder of Section 3.1 to be paired with any pooling head of Section 3.4.

Limitations of conjunctive pooling. Three properties of Equation (3)–Equation (5) restrict what the head can express, and each one motivates a family of pooling heads in Section 3.4. First, **a single attention gate is shared across classes**: the gate a_j is one scalar per time step, used unchanged for all C classes, so the head cannot represent a time step whose evidence is class-dependent – highly relevant to one class and irrelevant to another – because one value has to serve every class at once. Second, **the attention weights are not normalised over time**: the gate is a sigmoid, so the weights lie in $[0, 1]$ individually but do not sum to a fixed quantity, and the aggregation divides by T; the magnitude of $\hat{\mathbf{Y}}^k$ therefore depends on the length of the series, so two series carrying identical evidence but of different lengths do not receive comparable scores. This matters on the UCR archive, whose series lengths range from 15 to 2,844. Third, **the aggregation is a fixed mean, so evidence is diluted on long series**: Equation (5) averages over all T time steps with equal weight and that operator is fixed rather than learned, so when the evidence sits in a few decisive time steps of a long series those steps contribute a vanishing share of the mean and are swamped by the background. The mean cannot counteract this, and it cannot adapt to the datasets where the evidence is spread out instead.

2.4 Measuring the quality of an explanation

An importance map can look convincing and still be wrong, so explanation quality has to be measured. This report uses the two metrics of Early et al. (2024), computed with their implementation.

AOPCR: faithfulness of the explanation to the model. The Area Over the Perturbation Curve (Samek et al., 2017) tests whether the time steps an explanation calls important are the ones the model relies on. The steps are ranked from most to least important according to the model's own interpretation, masked one at a time in that order, and the confidence in the original prediction is tracked as masking proceeds; a faithful explanation produces a fast drop. Writing $\hat{\mathbf{Y}}^k(\mathcal{X})$ for the confidence in class k, O^k for the ranking induced by $\Phi_{k,\cdot}$, and $\mathcal{X}_j^{O^k}$ for the series with its j most important steps masked,

$$\mathrm{AOPC}(\mathcal{X}, O^k) \;=\; \frac{1}{T-1}\sum_{j=1}^{T-1}\Big[\hat{\mathbf{Y}}^k(\mathcal{X}) - \hat{\mathbf{Y}}^k\big(\mathcal{X}_j^{O^k}\big)\Big]. \tag{6}$$

AOPC on its own would reward a model whose confidence is simply fragile, so Early et al. (2024) add a random control. AOPCR repeats Equation (6) with three random orderings $R^{(1)}, R^{(2)}, R^{(3)}$ and reports the margin of the real ranking over chance:

$$\mathrm{AOPCR}(\mathcal{X}, O^k) \;=\; \frac{1}{3}\sum_{r=1}^{3}\Big[\mathrm{AOPC}(\mathcal{X}, O^k) - \mathrm{AOPC}\big(\mathcal{X}, R^{(r)}\big)\Big]. \tag{7}$$

A higher AOPCR means the ranking identifies evidence rather than noise. AOPCR needs no per-time-step ground truth, so it can be computed on every dataset in this study. One property governs how it may be used: it is built from raw confidence differences, so its scale follows the scale of the model's logits and has no fixed range. AOPCR values are comparable across models trained under an identical configuration, but not against AOPCR values reported elsewhere for models trained differently – Section 6.5 gives direct measured evidence of this.

NDCG@n: ranking quality of the explanation against ground truth. The Normalised Discounted Cumulative Gain (Järvelin & Kekäläinen, 2002) answers a different question: do the time steps that genuinely matter appear at the top of the ranking? That requires knowing which steps genuinely matter, which is available only for WebTraffic, where the generator records which steps it modified to create each class. Writing $\text{rel}_j \in \{0, 1\}$ for whether the step ranked j-th is one of the modified steps,

$$\text{NDCG@}n(O^k) \;=\; \frac{1}{\text{IDCG@}n} \sum_{j=1}^{n} \frac{\text{rel}_j}{\log_2(j+1)}, \tag{8}$$

where IDCG@n is the same sum evaluated on the ideal ranking. The normalisation gives NDCG@$n \in [0, 1]$, with 1.0 for a perfect ranking.

The two metrics are complementary. AOPCR measures *faithfulness* – agreement between the explanation and the model's own behaviour – and is available on all datasets but is unnormalised. NDCG@n measures *correctness* against an external ground truth and is normalised, but can only be computed on WebTraffic. Where the two disagree, NDCG@n is the stronger evidence, because it is checked against a known answer.

3 Proposed Methods

My contribution. The encoder family, the pooling heads and the training objective described in this section are my own design and implementation. They keep the two-stage MIL template of Section 2.3 – encoder, then pooling head – and the same tensor interface as MILLET, so that every comparison in Section 5 isolates one component at a time.

The architecture proposed here is called SEA-Net, for *Selective Evidence Aggregation Network*, because it combines multi-scale temporal feature extraction with selective, class-specific aggregation of the evidence found at each time step. It is introduced in this report and is not an existing model. A complete SEA-Net model is always the pair *encoder + pooling head*. Five **design goals**, fixed before implementation, motivate the choices that follow. The encoder has to be **small**, with substantially fewer parameters than InceptionTime and at a scale compatible with a microcontroller target. It has to be **at least as accurate** as that baseline under an identical training configuration. It has to be **length-preserving**, producing one embedding per time step at every stage, because the interpretation has one value per time step and AOPCR masks individual time steps, so T must never change inside the encoder. It has to be **local**: each embedding must depend on a bounded window of the input, otherwise the importance map becomes diffuse and uninformative. Finally, the design has to be **modular**, with the encoder and the pooling head selectable from a configuration file, so that any pair can be trained without modifying code.

3.1 The SEA-Net encoder

Figure 3 gives an overview of the encoder and of its interface with the pooling head; only the design choices behind it are discussed here.

The encoder is a stem convolution, which lifts the single input channel to d channels, followed by a stack of *multi-scale separable residual blocks* (Section 3.2). Three choices matter. Every convolution uses "same" zero padding with no stride and no pooling, so the time axis is preserved end to end and the encoder returns exactly the per-time-step embeddings $\mathbf{z}_j$ that Equation (1) requires, which is the **length-preserving** goal. The dilation grows geometrically from block to block, $\delta_i = \min(2^i, \delta_{\max})$, but is capped at $\delta_{\max} = 16$, so the receptive field widens quickly in the early blocks and then stops, keeping each embedding tied to a bounded window, which is the **local** goal. Finally, encoder and head communicate through one fixed contract: the encoder consumes $(B, 1, T)$ and returns (B, d, T), and the head returns the prediction (B, C) and the interpretation (B, C, T). Every variant and every head below respects that contract, which is what makes the combinations of Section 5 possible from configuration alone – the **modular** goal.

3.2 The multi-scale separable block

The block combines two established ideas with one addition. *Dilated* convolutions (Bai et al., 2018) space the filter taps apart, so the window widens without extra weights. *Depthwise-separable* convolutions (Chollet, 2017; Howard

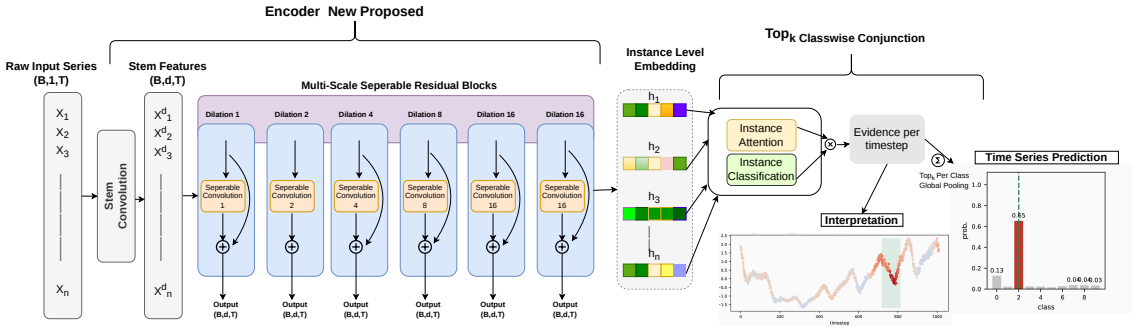


Figure 3: Overview of a complete SEA-Net model, read left to right, with the tensor shape under each stage. **Left, the SEA-Net encoder:** the series $(B, 1, T)$ passes through a stem convolution to (B, d, T) and then through six multi-scale separable residual blocks, whose dilation grows $1, 2, 4, 8$ and is then capped at 16; the curved arrow inside each block is the residual connection, and T is unchanged at every stage. **Middle:** one instance-level embedding $\mathbf{z}_j$ per time step. **Right, the MIL pooling head (Section 3.4):** an attention branch scores how much each time step counts, a classifier branch scores what it argues for, and their product is the evidence at that time step. Aggregating the evidence over time gives the prediction (bar chart); retaining the same per-time-step evidence gives the interpretation drawn on the series. The explanation and the prediction are the same quantities.

et al., 2017) factorise one convolution into a depthwise step $D_k^{(\delta)}$, which filters each channel independently along time, and a pointwise 1×1 step P, which mixes channels at one time step. The addition is that three kernel sizes run in parallel inside every block. One *unit* chains the two steps and one block runs two units and adds the input back:

$$\mathcal{U}(\mathbf{h}) = \mathrm{Drop}\Big(\mathrm{ReLU}\Big(\mathrm{BN}\Big(P\Big(\textstyle\sum_{k\in\mathcal{K}} D_k^{(\delta)}(\mathbf{h})\Big)\Big)\Big)\Big), \qquad \mathcal{K} = \{5, 11, 23\}, \tag{9}$$

$$\mathrm{Block}(\mathbf{h}) = \mathrm{ReLU}\big(\mathbf{h} + (\mathcal{U} \circ \mathcal{U})(\mathbf{h})\big). \tag{10}$$

Figure 4 draws both parts of this. Equation (9) examines the same signal at three time scales at once: a kernel of size 5 covers a narrow local pattern, a kernel of size 23 reaches about four times further, and their outputs are summed, so the block never has to commit to one scale in advance. The pointwise convolution P then combines what the three kernels found, BatchNorm (Ioffe & Szegedy, 2015) keeps activations on a stable scale, ReLU provides the non-linearity and dropout regularises. The residual connection in Equation (10) makes the block an identity plus a correction, which keeps deep stacks trainable. The lower panel of Figure 4 shows what the dilation buys: the same five taps of the $k = 5$ filter span five neighbouring time steps at $\delta = 1$ and 65 of them at $\delta = 16$, at no extra parameter cost.

Why the block is small. A standard convolution with d input and output channels and kernel size k costs d^2k weights, whereas a depthwise-separable pair costs only

$$\underbrace{dk}_{\text{depthwise}} + \underbrace{d^2}_{\text{pointwise}} \ll d^2k. \tag{11}$$

Counting the weights of Equation (9) at the wide setting $d = 128$, the three depthwise convolutions cost $d(6 + 12 + 24) = 5{,}376$ weights including one bias per channel, while the single pointwise convolution costs $d^2 + d = 16{,}512$ – three times more than all three depthwise convolutions combined. With BatchNorm's $2d$, one unit costs $22{,}144$ parameters, one block $44{,}288$, and the six-block encoder with its stem about $266{,}752$, consistent with the $269{,}083$ measured for the wide base model. A non-separable convolution covering the same three kernel sizes would cost $d^2(5{+}11{+}23) = 638{,}976$ weights for the same step, roughly $29\times$ more. Two consequences follow: the **small** goal becomes reachable at all, and the pointwise convolution now dominates the parameter count, which is what Section 3.3.1 attacks.

3.3 SEA-Net encoder variants

Seven variants are built on the same block, each presented with the problem it addresses, the modification in one equation, and its effect. Five are encoders in the strict sense; two are *wrappers* that sit in front of, or around, one of the encoders rather than replacing it. Every variant modifies the pipeline at one of the four points shown in Figure 5, and each diagram below is labelled with the point it uses.

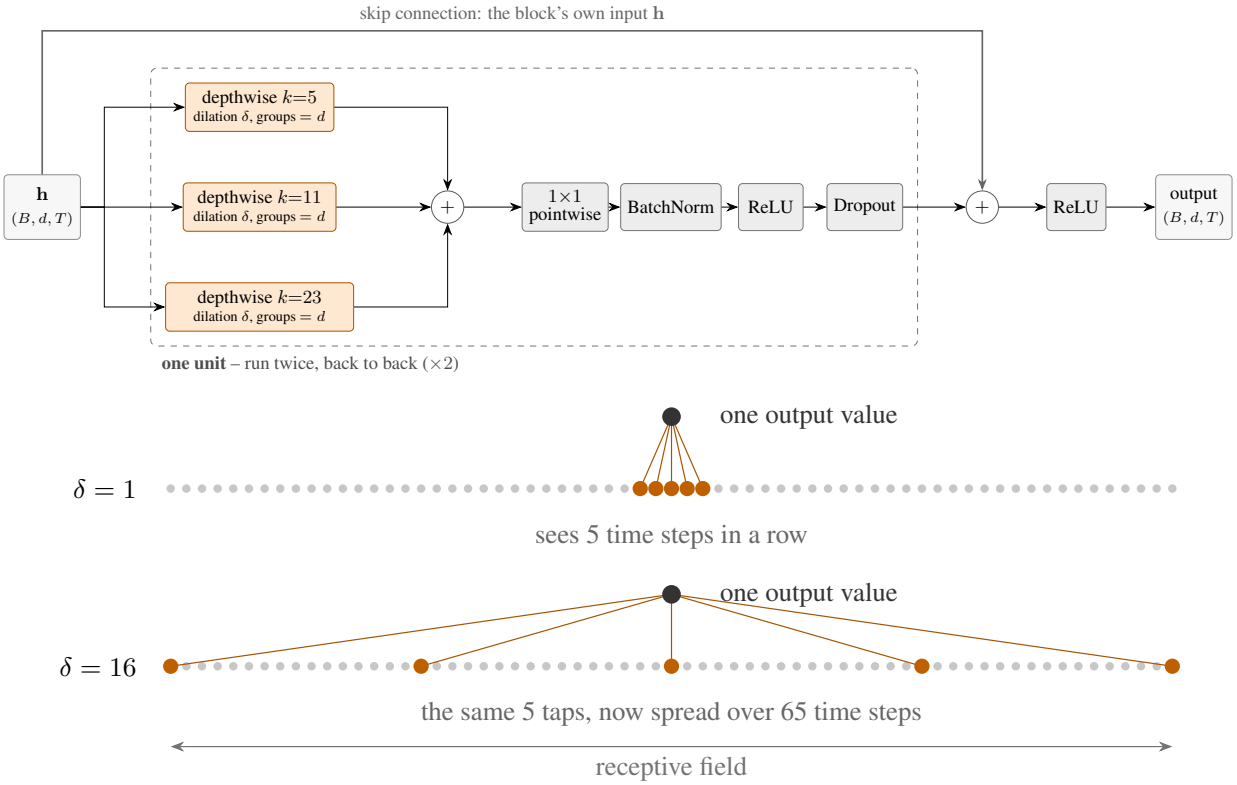


Figure 4: One multi-scale separable block. *Top:* the data path of one unit – three depthwise convolutions read the same input at kernel sizes 5, 11 and 23 and their outputs are summed, then a pointwise 1×1 convolution mixes the channels, followed by BatchNorm, ReLU and dropout. *Bottom:* the effect of dilation. Both rows use the same five taps of the $k = 5$ filter; at $\delta = 1$ they span five neighbouring time steps, at $\delta = 16$ the same five taps span 65.

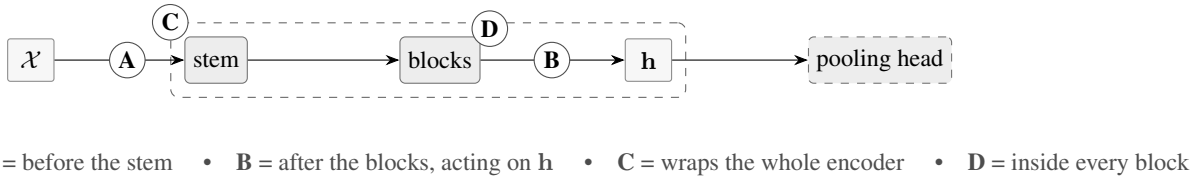


Figure 5: The reference SEA-Net pipeline and the four points at which an encoder variant can modify it. Every variant diagram below carries the letter of the point it uses.

3.3.1 SEA-Net bottleneck encoder

Problem. The pointwise step dominates the parameter count, costing d^2 weights per unit (Section 3.2).

Modification. Factorise that 1×1 convolution into two cheaper ones through a narrow middle layer of width $r = d/4$ – squeeze $d \to r$, then expand $r \to d$ (Figure 6):

$$P = P_{\text{expand}} P_{\text{squeeze}}, \qquad P_{\text{squeeze}} \in \mathbb{R}^{r \times d}, \quad P_{\text{expand}} \in \mathbb{R}^{d \times r}, \qquad \text{cost } d^2 \to 2dr. \tag{12}$$

Effect. With $r = d/4$ the pointwise cost becomes $d^2/2$, exactly half, independently of d. At the setting used in the experiments ($d = 64$, 4 blocks) this reduces the complete model from 57,580 to 41,324 parameters – a 28 % saving on the whole encoder, not only on the pointwise step. The block still consumes and returns (B, d, T), so the **length-preserving** and **modular** goals are unaffected. This is the smallest encoder built here and the one used in the headline result.

3.3.2 SEA-Net gated encoder

Problem. Not every feature channel is informative for every series, but the base encoder weights all channels equally.

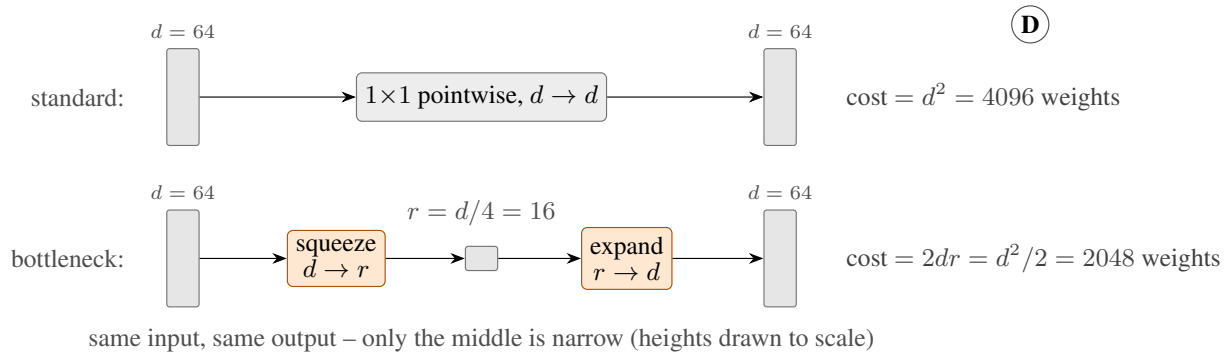


Figure 6: SEA-Net bottleneck encoder (point **D**). Top: the standard pointwise convolution. Bottom: the same input and output with a narrow middle layer. Heights are drawn to scale.

Modification. After the blocks have produced $\mathbf{h}$, collapse the time axis into a summary vector $\mathbf{s} \in \mathbb{R}^d$, convert it into a per-channel gate, and re-weight every time step with it:

$$\mathbf{s} = \text{summary}_j(\mathbf{h}), \qquad \mathbf{g} = \sigma(W_g\,\mathbf{s}) \in [0,1]^d, \qquad \mathbf{h}'_j = \mathbf{h}_j \odot (1+\mathbf{g}). \tag{13}$$

Effect. Two details are deliberate. summary_j is one of three ways of collapsing time – maximum, mean or last time step – and all three were trained under the same configuration so they could be compared. The multiplier is $(1+\mathbf{g}) \in [1,2]$ rather than $\mathbf{g}$: a plain multiplication would let the gate drive features towards zero, whereas amplifying on top of the original $\mathbf{h}$ preserves the differences between time steps, which are exactly what AOPCR and NDCG@n measure. The gate carries no time index, so it re-weights channels and never affects the time axis.

3.3.3 SEA-Net input-gated encoder

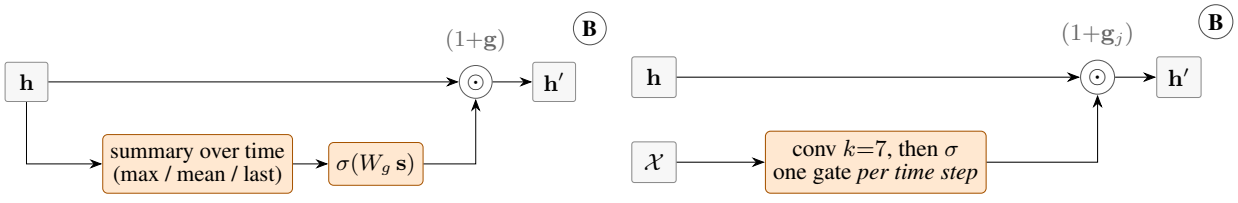
Problem. The gate of Equation (13) is one vector for the entire series, so it cannot express that a channel is informative in one part of the series and not in another.

Modification. Compute the gate directly from the raw input, so it varies with time:

$$\mathbf{g}_j = \sigma\big(W_g * \mathcal{X}\big)_j \in [0,1]^d, \qquad \mathbf{h}'_j = \mathbf{h}_j \odot (1+\mathbf{g}_j), \tag{14}$$

where $W_g * \mathcal{X}$ is a small convolution (kernel size 7) over the raw series.

Effect. The two variants differ only in the subscript: one gate for the whole series computed from deep features, against one gate per time step computed from the raw signal. Training both isolates the granularity of the gating mechanism, coarse against fine. Figure 7 draws the two side by side.



(a) SEA-Net gated encoder: one gate for the whole series, read from $\mathbf{h}$ (Equation (13)).

(b) SEA-Net input-gated encoder: one gate per time step, read from the raw input (Equation (14)).

Figure 7: The two gated SEA-Net encoders (point **B**). In both, the straight line through $\odot$ is the main path and the gate is a side branch, so neither can collapse the time axis. Orange marks what the variant adds, grey the unchanged encoder.

3.3.4 SEA-Net spike/trend encoder

Problem. A stack of convolutions followed by normalisation is biased towards smooth, sustained patterns, whereas some class evidence is a single sharp discontinuity.

Modification. Run an explicitly high-pass branch alongside the encoder and merge the two:

$$\text{spike}_j = \text{ReLU}\big(\,\text{BN}(\text{Conv}_3(\mathcal{X} - \text{avg}_w(\mathcal{X})))\big)_j \in \mathbb{R}^{d_{\text{spike}}}, \tag{15}$$

$$\mathbf{h}_j = P\big([\text{Encoder}(\mathcal{X})_j\,;\,\text{spike}_j]\big), \qquad \mathbf{h}'_j = \mathbf{h}_j \odot (1+\mathbf{g}) \;\text{ as in Equation (13).} \tag{16}$$

Effect. Subtracting a local moving average avg_w (window $w = 5$) leaves a signal that is near zero on smooth stretches and large only at discontinuities; a kernel-3 convolution turns it into $d_{\text{spike}} = 16$ channels, concatenated with the trend features and mixed back to width d. The expectation was an advantage on datasets with impulsive evidence. Table 9 shows this was the weakest of the five true encoders, so it was not met.

3.3.5 SEA-Net reconstruction-residual encoder

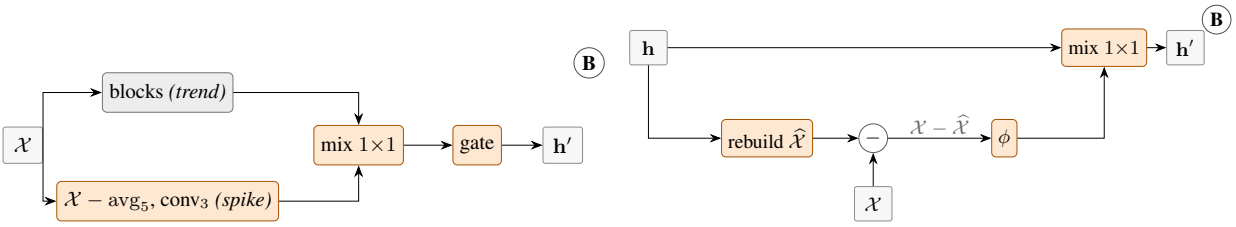
Problem. The previous variant assumes in advance what the encoder misses. This one measures it: which part of the input can the learned features *not* account for?

Modification. A 1×1 convolution reconstructs the raw input from the trend features; the reconstruction is subtracted from the real input and what remains becomes $d_{\text{res}} = 16$ additional channels:

$$\widehat{\mathcal{X}} = P_{\text{rec}}(\text{trend}), \qquad \mathbf{h}_j = P\big([\,\text{trend}_j\,;\,\phi(\mathcal{X} - \widehat{\mathcal{X}})_j\,]\big), \tag{17}$$

with ϕ the same small convolution, BatchNorm and ReLU used for the spike branch.

Effect. The residual $\mathcal{X} - \widehat{\mathcal{X}}$ marks the parts of the signal the smooth features cannot explain, a principled place to look for an unusual event. There is no separate reconstruction loss: the branch is trained end to end, so it is a learned residual feature rather than an auto-encoder. It was the second strongest encoder in Table 9. Figure 8 shows the two two-branch encoders together.



(a) SEA-Net spike/trend encoder: a high-pass branch off the raw input (Equation (15)).

(b) SEA-Net reconstruction-residual encoder: a branch off $\mathbf{h}$ (Equation (17)).

Figure 8: The two two-branch SEA-Net encoders (point **B**): the spike/trend encoder branches off the raw input to capture discontinuities, the reconstruction-residual encoder branches off the features to capture what they failed to rebuild.

3.3.6 SEA-Net multi-scale channels (wrapper)

Problem. The stem receives a single raw channel, so any local statistic it needs must be learned from scratch.

Modification. Append rolling maximum, minimum and standard deviation over several windows, plus the sign of the first difference, as extra input channels:

$$\mathcal{X}^{+} = \Big[\,\mathcal{X}\,;\;\big(\text{roll\_max}_w(\mathcal{X})\big)_{w\in\mathcal{W}}\;;\;\big(\text{roll\_min}_w(\mathcal{X})\big)_{w\in\mathcal{W}}\;;\;\big(\text{roll\_std}_w(\mathcal{X})\big)_{w\in\mathcal{W}}\;;\;\text{sign}(\Delta\mathcal{X})\,\Big], \tag{18}$$

with $\mathcal{W} = \{3, 7, 15, 31\}$ (all odd, so each window stays centred).

Effect. This is a wrapper, not an encoder: it widens the input from 1 to $1 + 4 \cdot 3 + 1 = 14$ channels before any encoder above. Maximum, minimum and standard deviation are used rather than a rolling mean deliberately – a mean is linear, so the stem convolution could learn it on its own, whereas the other three are non-linear and add information a convolution cannot compute. A BatchNorm normalises the 14 channels first, because they do not share the scale of the raw series; omitting it cost accuracy.

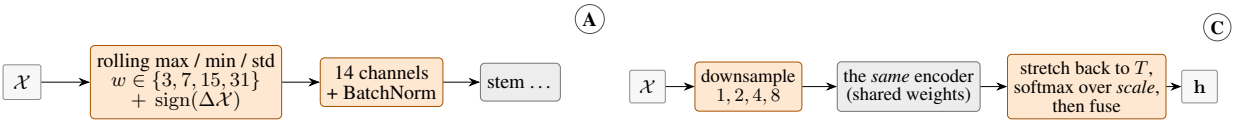
3.3.7 SEA-Net multi-scale pyramid (wrapper)

Problem. The dilation cap bounds the receptive field, so evidence visible only over a long span cannot be reached.

Modification. Run the same encoder on the series downsampled by 1, 2, 4 and 8, stretch the outputs back to length T, and fuse them with attention over the scale axis:

$$\mathbf{h}_j = \sum_{s \in \mathcal{S}} \alpha_j^s \, \mathbf{h}_j^{(s)}, \qquad \alpha_j^s = \mathrm{softmax}_s \left(\mathrm{score}(\mathbf{h}_j^{(s)}) \right), \qquad \mathcal{S} = \{1, 2, 4, 8\}. \tag{19}$$

Effect. The softmax normalises over *scale* and never over time, so the per-time-step signal AOPCR and NDCG@n depend on is preserved, and one set of encoder weights is shared across all scales, so the parameter count barely changes and the extra cost is compute (about $1.9\times$). This is the one-dimensional analogue of multi-magnification whole-slide-image MIL (Hashimoto et al., 2020), and it was the weakest variant built (Table 9). It also imposes a constraint: the wrapped encoder must be shallow enough that its receptive field does not exceed $T / \max(\mathcal{S})$, or the coarsest scales collapse to a nearly constant signal. The pyramid therefore uses a shallow two-block base, and the pipeline verifies this condition automatically. Figure 9 shows where each wrapper attaches.



(a) SEA-Net multi-scale channels: widens the input before the stem (Equation (18)).

(b) SEA-Net multi-scale pyramid: runs one shared encoder at four scales (Equation (19)).

Figure 9: The two SEA-Net wrappers. They are not encoders: the channel wrapper acts before the stem (point **A**) and the pyramid wrapper around the whole encoder (point **C**), and both delegate feature extraction to one of the encoders above.

Alternatives considered and rejected. A **Transformer encoder** was rejected on two grounds: full self-attention costs $\mathcal{O}(T^2)$ in time and memory, prohibitive for series of 1,000–1,500 points on the target hardware, and unrestricted attention lets any output position draw on any input position, contradicting the **local** goal. **Pruning or quantising InceptionTime** would reduce size but not address the structural problem of Section 2.1: the model still ends in global average pooling, so it would still require a post-hoc explainer – which is also why **keeping a large backbone and explaining it afterwards** was rejected (Section 1.3).

3.4 MIL POOLING HEADS

My contribution. The seven pooling heads below were designed and implemented by me. Five (Section 3.4.1–Section 3.4.5) follow directly from the three limitations of conjunctive pooling identified in Section 2.3. The last two adapt mechanisms published for whole-slide image classification (Ilse et al., 2018; Li et al., 2021); the transfer to time series, the per-class formulation and the implementation are mine.

Every proposed head keeps the two-part template of conjunctive pooling: an attention branch that decides *where* to look, and a per-time-step classifier that decides *what* each position argues for,

$$\mathbf{p}_j = W_c \mathbf{z}_j + \mathbf{b}_c \in \mathbb{R}^C, \qquad \mathbf{a}_j = \mathcal{A}(\mathbf{z}_j), \tag{20}$$

so a head is fully specified by two choices: the shape of the attention branch $\mathcal{A}$ (slot 1) and the operator combining the T evidence values g_j^k into $\hat{\mathbf{Y}}^k$ (slot 2). Only slot 2 distinguishes five of the seven heads: they keep the class-wise attention branch and change how the T evidence values are reduced to one number – a plain mean, a smooth weighting, a hard selection of the largest few, or a blend with the maximum. The two diagrams that draw the shared template and compare the seven aggregators side by side on the same evidence profile are in the repository documentation (Section 4.2).

For every head, Φ is not produced by a separate branch: it is exactly the per-time-step, per-class quantity that $\hat{\mathbf{Y}}$ is built from – Equation (22), for instance, sets $\Phi_{k,j} = g_j^k$, the very term that is averaged to obtain $\hat{\mathbf{Y}}^k$. The implementation follows the same rule, deriving both outputs from one intermediate tensor. Computing them independently would allow them to disagree, which is precisely the weakness of post-hoc explainers (Section 1.3).

The heads are presented in the order of the limitations they address (Section 2.3): Section 3.4.1 takes on the shared attention gate, Section 3.4.2 the unnormalised weights, and Section 3.4.3–Section 3.4.7 the fixed mean, through five different aggregation strategies.

3.4.1 CLASS-WISE CONJUNCTIVE POOLING

Limitation addressed: the shared attention gate. The single gate of Equation (3) must serve all classes at once.

Change. Produce one gate per class from the same attention network, giving it C outputs instead of one:

$$\mathbf{a}_j = \sigma(W_2 \tanh(W_1 \mathbf{z}_j)) \in [0,1]^C, \qquad W_1 \in \mathbb{R}^{d_a \times d},\ W_2 \in \mathbb{R}^{C \times d_a}, \tag{21}$$

$$g_j^k = a_j^k\, \mathbf{p}_j^{\,k}, \qquad \hat{\mathbf{Y}}^{\,k} = \frac{1}{T}\sum_{j=1}^{T} g_j^k, \qquad \Phi_{k,j} = g_j^k. \tag{22}$$

The cost is $(C-1)(d_a+1)$ extra weights, a widening of the last attention layer only.

Property 3.1 (Strict generalisation)**.** If $a_j^k = a_j$ for every class k, Equation (22) reduces exactly to conjunctive pooling Equation (5); if $a_j^k \to 1$ it reduces to instance pooling.

Expected benefit. A time step can now support one class and simultaneously argue against another, making the class-specific interpretation meaningful. By Property 3.1 the head can only add capacity: in the worst case it reproduces the baseline exactly.

3.4.2 Softmax conjunctive pooling with a learnable sharpness

Limitation addressed: the unnormalised weights. The sigmoid gate makes the bag score depend on the series length.

Change. Replace the sigmoid gate by a distribution over time, normalised by a softmax whose sharpness is controlled by a learnable parameter $\tau > 0$:

$$\alpha_j^k = \mathrm{softmax}_j\left(\frac{s_j^k}{\tau}\right) = \frac{\exp(s_j^k/\tau)}{\sum_{i=1}^{T}\exp(s_i^k/\tau)}, \qquad s_j^k = \left(W_2 \tanh(W_1 \mathbf{z}_j)\right)_k, \tag{23}$$

$$\hat{\mathbf{Y}}^{\,k} = \sum_{j=1}^{T} \alpha_j^k\, \mathbf{p}_j^{\,k}, \qquad \Phi_{k,j} = \alpha_j^k\, \mathbf{p}_j^{\,k}. \tag{24}$$

The parameter τ divides the scores before the softmax and so controls how concentrated the resulting weights are: a large τ flattens them towards uniform, a small τ concentrates them on the highest-scoring steps. It is stored as $\tau = \exp(\rho)$ with ρ learnable, so it stays strictly positive, and adds one weight.

Property 3.2 (Interpolation between mean and max)**.** As $\tau \to \infty$, $\alpha_j^k \to 1/T$ and Equation (24) becomes a plain mean over time; as $\tau \to 0$ it selects the single highest-scoring time step. Because the weights sum to one, $\hat{\mathbf{Y}}^{\,k}$ does not depend on T.

Expected benefit. Series of different lengths become comparable, which should matter most on the UCR archive, and the model learns how concentrated its attention should be.

3.4.3 Adaptive class-wise pooling

Limitation addressed: the fixed mean. It is appropriate only when the evidence is spread out; where it is a single short event a max-like aggregator is better, and no fixed operator can be right for both.

Change. Keep the per-class gates of Equation (21) and aggregate with a learnable sharpness $\beta_k \geq 0$ per class, as a numerically stable soft-argmax:

$$\beta_k = \mathrm{softplus}(\theta_k), \quad w_j^k = \mathrm{softmax}_j\left(\beta_k\, g_j^k\right), \quad \hat{\mathbf{Y}}^{\,k} = \sum_{j=1}^{T} w_j^k\, g_j^k, \quad \Phi_{k,j} = g_j^k. \tag{25}$$

It adds C weights over class-wise pooling.

Property 3.3 (Learned mean–max aggregator)**.** $\beta_k \to 0$ gives $w_j^k \to 1/T$, that is, class-wise conjunctive pooling; $\beta_k \to \infty$ gives $\hat{\mathbf{Y}}^{\,k} \to \max_j g_j^k$. Since $\hat{\mathbf{Y}}^{\,k}$ is always a convex combination of the g_j^k, it stays within their range, which keeps training stable.

Expected benefit. The aggregation adapts per class and per dataset instead of being fixed. β_k is initialised small (0.3), so training starts close to the safe class-wise mean and the optimiser sharpens it only where that reduces the loss.

3.4.4 TOP-k CONJUNCTIVE POOLING

Limitation addressed: the fixed mean, directly. On a series of hundreds of time steps most positions are background, and averaging over all of them dilutes the few decisive ones.

Change. Average only the k largest evidence values of each class, with k a fixed fraction κ of the series length:

$$k = \max\big(1, \lceil \kappa T \rceil\big), \qquad \mathcal{S}^k_{\text{top}} = \big\{\, j : g^k_j \text{ is among the } k \text{ largest of } (g^k_1, \ldots, g^k_T) \,\big\}, \tag{26}$$

$$\hat{\mathbf{Y}}^{\,k} = \frac{1}{k} \sum_{j \in \mathcal{S}^k_{\text{top}}} g^k_j, \qquad \Phi_{k,j} = g^k_j. \tag{27}$$

$\kappa \in (0, 1]$ is a hyperparameter rather than a weight, so this head adds no parameters over class-wise pooling.

Property 3.4 (Strict generalisation)**.** $\kappa = 1$ recovers class-wise conjunctive pooling exactly. For $\kappa < 1$ the gradient reaches only the selected time steps, which concentrates the evidence.

Expected benefit. The prediction rests on a small, explicit set of time steps, so masking exactly those steps should damage the model far more than masking the same number of random ones – an improvement in explanation quality rather than in accuracy. Table 10 tests Property 3.4 by varying κ, and this is the head used in the best models of Section 5.

3.4.5 ATTENTION-MAX POOLING

Limitation addressed: the fixed mean, as a control. Adaptive class-wise pooling also removes the fixed mean, but its soft-argmax still mixes in every time step to some degree. This head is the simplest hard alternative, to test whether the smoother learned aggregator is justified.

Change. A learnable per-class blend of the mean and the maximum, adding C weights:

$$\hat{\mathbf{Y}}^{\,k} = (1 - \lambda_k)\, \frac{1}{T} \sum_{j=1}^{T} g^k_j \;+\; \lambda_k \max_j g^k_j, \qquad \lambda_k = \sigma(\theta_k) \in [0, 1]. \tag{28}$$

Property 3.5. $\lambda_k \to 0$ recovers class-wise conjunctive pooling; $\lambda_k \to 1$ gives max pooling. Each class learns independently whether its evidence is sustained or impulsive.

Expected benefit. Equation (28) and Equation (25) use the same per-class gate and differ only in how they reduce it to one number, so comparing them isolates a single factor: soft against hard selection.

3.4.6 PER-CLASS GATED ATTENTION

Existing work (reused). The gated attention mechanism is due to Ilse et al. (2018) and is used in the multi-scale MIL pipeline of Hashimoto et al. (2020), where a bag is a whole-slide image and an instance is an image patch (Section 2.2).

Limitation addressed: the shared gate and the fixed mean, through a stronger attention branch. The baseline gate is a single $\tanh$ layer, which limits how selective it can be.

Change. Transfer gated attention to time series (bag = series, instance = time step) and make it *per class*, which the original formulation is not:

$$\mathbf{A}_j = \tanh(V \mathbf{z}_j) \odot \sigma(U \mathbf{z}_j) \in \mathbb{R}^{d_a}, \qquad V, U \in \mathbb{R}^{d_a \times d}, \tag{29}$$

$$\alpha^k_j = \operatorname{softmax}_j\big((W \mathbf{A}_j)_k\big), \qquad \hat{\mathbf{Y}}^{\,k} = \sum_{j=1}^{T} \alpha^k_j\, \mathbf{p}^{\,k}_j, \qquad \Phi_{k,j} = \alpha^k_j\, \mathbf{p}^{\,k}_j. \tag{30}$$

The $\tanh$ branch proposes candidate attention features and the σ branch decides which to admit, so a feature is used only where the second branch judges it reliable. The cost is a second $d \to d_a$ layer, making this the most expensive proposed head.

Expected benefit and a structural caveat. A more selective gate should sharpen the interpretation. However, unlike every other head here, this one has no blend parameter that recovers conjunctive pooling, so no setting of its weights returns the baseline. Section 6.4 shows that this matters in practice.

3.4.7 DUAL-STREAM CONJUNCTIVE POOLING

Existing work (reused). The two-stream mechanism is due to Li et al. (2021) (DSMIL), again for whole-slide images: one stream aggregates all instances, a second focuses on the most critical instance and on the instances resembling it (Section 2.2).

Limitation addressed: the fixed mean, when the evidence is scattered. A single decisive time step should be able to dominate without the surrounding evidence being discarded.

Change. I removed the contrastive pre-training of Li et al. (2021), made the critical instance and the query per class, and combined the two streams with a learnable blend starting at the conjunctive baseline. Stream A is the class-wise conjunctive mean of Equation (22); stream B locates the most supporting time step for each class, embeds a query for every time step, and re-weights by similarity to that critical query:

$$m_k = \arg\max_j \; g_j^k, \qquad \mathbf{q}_j = W_q \mathbf{z}_j \in \mathbb{R}^{d_a}, \qquad U_j^k = \mathrm{softmax}_j(\langle \mathbf{q}_j, \mathbf{q}_{m_k} \rangle), \tag{31}$$

$$\hat{\mathbf{Y}}^{\,k} = (1-\lambda)\,\frac{1}{T}\sum_{j=1}^{T} g_j^k + \lambda \sum_{j=1}^{T} U_j^k\, \mathbf{p}_j^{\,k}, \qquad \Phi_{k,j} = (1-\lambda)\, g_j^k + \lambda\, U_j^k \mathbf{p}_j^{\,k}, \tag{32}$$

with a single learnable scalar $\lambda = \sigma(\theta)$, initialised at 0.05.

Property 3.6 (Safe by construction)**.** $\lambda \to 0$ recovers class-wise conjunctive pooling. The interpretation uses the same blend as the logit, so the explanation always matches the decision. Attention is computed only against the single critical time step, so the cost is $\mathcal{O}(T d_a C)$ rather than $\mathcal{O}(T^2)$.

Expected benefit. Property 3.6 suggests this head should never be worse than the baseline, and it was designed specifically to improve AOPCR. Section 6.4 shows it is in fact the weakest head tested, and explains why a safety property that nothing in the training objective drives the model towards is ineffective in practice.

3.5 TRAINING OBJECTIVE

Every model in this report is trained with the same objective, which has two terms:

$$\mathcal{L} = \underbrace{\mathrm{CE}(\hat{\mathbf{Y}}, y)}_{\text{classification}} + \lambda_{\text{focus}} \cdot \underbrace{\Big(-\sum_{j=1}^{T} \bar{a}_j \log(\bar{a}_j + \epsilon) \Big)}_{\text{attention entropy}}. \tag{33}$$

Classification term. Cross-entropy between the bag logits and the true label, with *label smoothing* at 0.13, which replaces the one-hot target by one that puts $1-\varepsilon$ on the correct class and spreads ε over the others. This prevents the model from driving its logits to arbitrarily large values in order to reach a probability of exactly one. That matters more here than in an ordinary classifier, because the interpretation is built from those same logits: over-confident logits produce importance maps whose scale is dominated by a few saturated values.

Attention entropy term. The entropy of the attention distribution returned by the pooling head, where $\bar{a}_j$ is that attention averaged over classes (so $\bar{a}_j = \frac{1}{C}\sum_k a_j^k$ for the class-wise family) and ϵ avoids $\log 0$. Entropy measures how spread out a distribution is: attention concentrated on a few time steps has *low* entropy, attention spread evenly over the series has *high* entropy. Adding entropy to a loss that is minimised therefore pushes the model towards concentrated attention.

This term exists for the interpretation, not for the classification. Without it, nothing prevents the model from reaching a correct prediction using a small amount of attention spread over hundreds of time steps; the prediction is then correct but the importance map is flat, and a reader cannot tell where the evidence stops. With the term, the model must reach the same prediction using fewer time steps, which is what makes the resulting map readable and what both AOPCR and NDCG@n reward. The trade-off was measured directly: Table 11 shows that switching the term off improves accuracy by about one point while reducing both explanation metrics. The weight is $\lambda_{\text{focus}} = 0.01$ for every SEA-Net model and 0 for the MILLET baseline, since the term is a SEA-Net addition.

Table 2: The two dataset families used in this study.

Dataset	#series	Length	#classes	Per-step ground truth?
WebTraffic	1,000 (500 / 500)	1,008	10	yes
UCR (85 published)	40–16,637	24–2,709	2–60	no
UCR (128 total)	40–24,000	15–2,844	2–60	no

4 Experimental Methodology

4.1 Datasets

WebTraffic (Early et al., 2024) is synthetic, so its generator records which time steps it modified to create each class – the last column of Table 2, and the reason WebTraffic is the only dataset on which NDCG@n can be computed; on the UCR archive (Dau et al., 2019) only AOPCR is available. WebTraffic is also small and fast to train on, so it was the screening set: every new encoder and pooling head was evaluated there first and only the strongest combinations went on to the full UCR experiments, a choice whose cost Section 5.2 quantifies. Early et al. (2024) published MILLET results on 85 of the 128 UCR datasets, so those 85 make a dataset-by-dataset comparison possible; the remaining 43 are run as well so that no conclusion rests on one convenient subset.

4.2 Baseline, training configuration and pipeline

The baseline is MILLET's own model: an **InceptionTime encoder with conjunctive pooling**, 423,707 parameters. Re-trained here under the configuration of Table 3 it reaches **0.887 accuracy** on WebTraffic, and that re-trained value is the one every comparison in this report uses. The SEA-Net models are trained under exactly the same configuration, with only the encoder and the pooling head changed – the smallest of them being the SEA-Net bottleneck encoder with Top-k conjunctive pooling at 41,324 parameters. FCN and ResNet (Wang et al., 2017), each with the same conjunctive head, were re-trained the same way as secondary reference points.

Re-training the baseline rather than quoting it is the central methodological decision of this work. A published accuracy combines the effect of the architecture with the effect of the training configuration its authors used, and the two cannot be separated after the fact. Under one identical configuration they can: any difference in Section 5 is then attributable to the model alone. The price is that the numbers in this report are not directly comparable with the ones MILLET published, and that comparison is a separate question with its own answer, so it is kept out of the main results and treated on its own in Section B.4.

Table 3: Training configuration, identical for every model in this study. The complete hyperparameter list, including the head-specific initial values, is in Table 14.

Setting	Value
Encoder size (every SEA-Net model)	width $d = 64$, 4 blocks; 41,324–69,768 parameters
Optimiser	Adam (Kingma & Ba, 2015), lr $= 1.25 \times 10^{-3}$, weight decay $= 1.2 \times 10^{-4}$
Epochs / early stopping	max 400 epochs; stop after 60 epochs with no improvement
Batch size	$\min(\lfloor n_{\text{train}}/10 \rfloor, 16)$, floor of 2
Validation split	stratified 80/20 when $n_{\text{train}} \geq 100$; else train-loss early stopping
Label smoothing	0.13
Focus penalty λ_{focus}	0.01 (Equation (33)); 0 for the MILLET baseline
Seeds	3 for the four models in Table 12; 1 for the rest
Hardware	Grid'5000 GPU nodes (Lille, Sophia)

Two settings adapt to the dataset rather than being fixed, because the UCR archive spans 40 to 24,000 series: the validation split is held out only when a dataset has at least 100 training series, and the batch size scales with the training set. The number of seeds also changed during the internship: the initial experiments were single-seed, because 72 variants over 129 datasets was already close to the available compute budget (Section A.5), and two further seeds were added afterwards for the four models on which the main claims rest, so variance estimates exist exactly where the claims are made. Classification is measured by accuracy and test loss, explanation quality by AOPCR and NDCG@n (Section 2.4), and cost by parameter count, saved model size and time per prediction; over the UCR archive the mean accuracy is reported together with the mean accuracy rank across datasets, following Demšar (2006), because a mean over datasets as different as these can be dominated by a few of them while a rank cannot.

Table 4: The ten best models on WebTraffic, ranked by accuracy, with the re-trained baselines below. Every model is written as *encoder + pooling head*; the corresponding configuration identifiers, and the other 62 models, are in the full leaderboard published with the code (Section 4.2). Starred rows were trained with 3 seeds and show the mean, with the per-seed values in Table 12; all other rows are a single seed. Higher is better for accuracy, AOPCR and NDCG@n, lower for parameters. AOPCR is not normalised (Section 2.4), so it compares rows of this table with each other and never with a value printed in another paper.

#	Model (encoder + pooling head)	Accuracy	AOPCR	NDCG@n	Params
1	SEA-Net gated (mean) + Top-k^*	**0.955**	2.225	0.750	61,740
2	SEA-Net base + conjunctive (MILLET head)	0.954	1.502	0.698	269,083
3	SEA-Net gated (max) + Top-k	0.952	2.268	0.719	61,740
4	SEA-Net spike/trend + Top-k	0.950	2.316	0.756	67,020
5	SEA-Net multi-scale channels (input-gated base) + Top-k	0.948	2.316	0.757	69,768
6	SEA-Net multi-scale channels (gated base) + Top-k	0.948	2.223	0.717	67,592
7	SEA-Net bottleneck + Top-k^*	0.947	2.621	**0.773**	**41,324**
8	SEA-Net reconstruction-residual + adaptive class-wise	0.946	2.601	0.768	62,935
9	SEA-Net input-gated + Top-k	0.946	**2.803**	0.748	58,092
10	SEA-Net base + class-wise conjunctive	0.946	1.722	0.686	269,164
Also discussed in this report					
	SEA-Net input-gated + adaptive class-wise*	0.905 ± 0.030	2.651 ± 0.437	0.732	58,102
Baselines re-trained here, identical training configuration					
	MILLET (InceptionTime + conjunctive)*	0.887 ± 0.010	2.569 ± 0.884	0.677	423,707
	ResNet + conjunctive	0.772	2.952	0.554	506,331
	FCN + conjunctive	0.742	3.826	0.533	267,035

My contribution. The experimental pipeline is my own work, and it is what made 72 variants over 129 datasets possible within the internship. A model is always *encoder + pooling head*, both selected by name in a small YAML file rather than hardcoded, so every additional row in the results tables is a new configuration file and not a new script. Each model writes into its own results directory, one row per dataset per seed, runs are resumable after a job hits the cluster wall-clock limit, and every figure and table in this report is regenerated from the stored results by a single command – which is why no number reported here was transcribed by hand. The stack is PyTorch (Paszke et al., 2019), MLflow for tracking and Optuna (Akiba et al., 2019) for the hyperparameter search.

Supplementary material. To keep this report readable, the material that supports the results without being needed to follow them is published with the code rather than printed here: the complete 72-model leaderboard, the encoder × pooling head grid, the per-dataset UCR results, the ensemble and Optuna experiments, the win/tie/loss and accuracy-versus-length analyses, the pooling-head diagrams, the exact commands and the software environment. All of it, together with the implementation, is in the repository README at `https://github.com/ubaidur404786/sea-net`, on branch `main`, which is the branch matching this report.

5 Results

Seventy-two model variants were compared: the SEA-Net encoders paired with the proposed pooling heads and with MILLET's heads, plus the FCN, ResNet and InceptionTime baselines re-trained here, each run on WebTraffic and on the UCR archive – up to 129 datasets per model. Every number in this section was produced here, under the single configuration of Table 3; no accuracy is copied from another paper. Rows marked with a star were trained with 3 seeds and report the mean, every other row is a single seed (Section 4.2).

5.1 Main comparison on WebTraffic

WebTraffic is the only dataset on which classification quality and explanation correctness can both be measured (Section 4.1), which makes Table 4 the principal table of this report.

Four observations follow from Table 4. First, all ten leading models are SEA-Net models, and each of them exceeds the best re-trained baseline (0.887). Second, the top ten span only nine thousandths of accuracy, from 0.946 to 0.955, which is why Section 6.5 declines to name a winner on accuracy alone. Third, the parameter column separates models that the accuracy column does not: ranks 2 and 10 require about 269 K parameters to reach the accuracy that rank 7 attains with 41 K. Fourth, and this is the central result, SEA-Net bottleneck + Top-k (rank 7) has both the best NDCG@n of the table (0.773) and the smallest parameter count, so no classification or explanation quality was traded for the reduction in size. This is what Top-k pooling was designed to achieve (Section 3.4.4): by averaging only

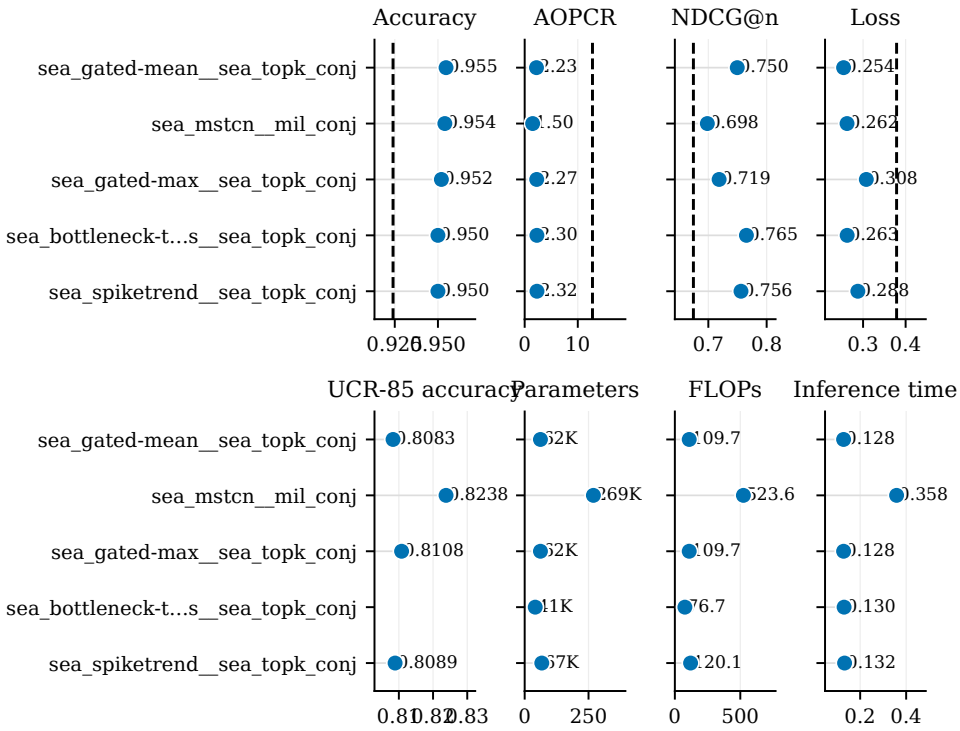


Figure 10: The five most accurate models on WebTraffic across every metric measured – all five are SEA-Net models. One row is one model and the row order is identical in every panel, so a model can be followed from the quality panels (accuracy, AOPCR, NDCG@n, loss) to the cost panels (UCR-85 accuracy, parameters, compute, prediction time). The contrast between the two blocks is the point: the models at the top of the quality panels sit at the inexpensive end of the cost panels. The clearest case is the second row on accuracy, the SEA-Net base encoder with MILLET's conjunctive head, which needs roughly six times the parameters of SEA-Net bottleneck + Top-k while being five thousandths more accurate.

the k most decisive time steps, the interpretation is exactly zero wherever the model used no evidence, whereas the baseline's importance curve never quite reaches zero and its extent has to be guessed (Figure 1).

The reduction holds on the quantities a device constrains. Measured on the real WebTraffic input, SEA-Net bottleneck + Top-k uses 90 % fewer parameters than the re-trained MILLET baseline, its saved file shrinks from 4.11 MB to 1.43 MB, and one prediction is about 1.6 times faster; the file size is what matters for the microcontroller target. Table 7 gives the full cost comparison, including its counterpart: the SEA-Net models take two to three times *longer to train*. Figure 10 places quality and cost side by side for the five most accurate models.

5.2 Accuracy across the 85 UCR datasets

A single dataset cannot support a general claim, so the same models were evaluated on the 85 UCR datasets for which MILLET results are published. Table 5 reports the mean accuracy with the mean rank beside it, because a mean alone can be dominated by a few datasets (Section 4.2).

This table tells a different story from Table 4. Over the UCR archive the re-trained MILLET baseline is ahead of all three SEA-Net models, on the mean and on the rank. This does not invalidate the WebTraffic result but it bounds it: both headline models were selected because they performed well on WebTraffic (Section 4.1), and one dataset does not predict behaviour on 85 heterogeneous ones. SEA-Net remains competitive – within one to two accuracy points of the baseline at about a tenth of the parameters – but the results do not support a claim of general superiority.

Finally, neither component alone accounts for the result. Averaged over every partner, the encoder choice moves WebTraffic accuracy over a range of 0.858–0.937 and the pooling head choice over 0.744–0.921, two spreads of comparable size, and the best pair is not composed of the two individually best halves (Section B.2).

Table 5: Accuracy over the 85 UCR datasets, for models re-trained here under the configuration of Table 3. "Mean rank" is the average accuracy rank over the 84 datasets shared by the 28 models that completed the whole archive, so a lower value is better. How these numbers relate to the accuracies MILLET published is a separate question and is treated in Section B.4.

Model	Mean accuracy	Mean rank
SEA-Net gated + Top-k	0.8083	15.29
SEA-Net bottleneck + Top-k	0.8097	15.40
SEA-Net input-gated + adaptive class-wise	0.8153	15.47
ResNet + conjunctive	0.8146	13.54
FCN + conjunctive	0.8141	14.48
MILLET (InceptionTime + conjunctive), re-trained here	**0.8274**	**12.60**

6 Discussion

6.1 Why the parameter reduction did not cost accuracy

The bottleneck encoder removes parameters from one specific place, and the results indicate it was the right place. After a convolution is split into a depthwise and a pointwise step, the pointwise step dominates: at $d = 128$ it costs three times as much as all three depthwise convolutions combined (Section 3.2). The depthwise step looks along time; the pointwise step only mixes channels. Equation (12) halves the second and leaves the first untouched, which is why the encoder can lose 28 % of its weights without losing the mechanism that extracts temporal patterns. What the results add is that most of the channel-mixing capacity was not being used productively: SEA-Net bottleneck + Top-k matches or exceeds the re-trained MILLET baseline on accuracy, AOPCR and NDCG@n simultaneously with 90 % fewer parameters (Table 4). That is a claim about the baseline's capacity as much as about the proposed encoder, and the parameter arithmetic alone could not have predicted it.

6.2 Why Top-k pooling improved the explanations

Top-k pooling produced the best NDCG@n of every model with complete seed data, and the mechanism can be identified rather than inferred. Averaging only the κT largest evidence values makes $\Phi_{k,j}$ exactly zero everywhere the model did not use, instead of a small weight that never quite reaches zero; since NDCG@n scores whether the truly modified steps appear at the top of the ranking, removing the background from that ranking is directly what the metric rewards.

The κ ablation (Table 10) confirms this and bounds it. NDCG@n is highest at $\kappa = 0.1$ (0.777) and falls monotonically as κ grows towards 1 (0.722, the value of class-wise conjunctive pooling), exactly the behaviour the design predicts. Accuracy is not monotone: it peaks between $\kappa = 0.1$ and 0.25 and collapses to 0.888 at $\kappa = 0.05$, so selecting too few time steps discards evidence the classifier needs and the mechanism has an optimum rather than improving without limit. The complementary effect appears in the attention entropy term: switching it off raises accuracy from 0.938 to 0.950 while lowering AOPCR from 2.778 to 2.303 and NDCG@n from 0.777 to 0.765 (Table 11). Both experiments point the same way – concentrating the evidence improves the explanation and costs about one accuracy point.

6.3 Encoder and pooling head interact

The best model of the study, SEA-Net gated + Top-k, is not built from the individually best encoder and the individually best pooling head: averaged over their pairings, the gated encoder reaches 0.902 and Top-k pooling 0.904, both mid-table (Table 9, Table 8). Two ordinary components producing the best combination is evidence of an interaction. A plausible mechanism is that the gated encoder's channel gate (Equation (13)) already suppresses the channels carrying no signal, making a hard-selection head the natural partner, whereas a smoother head would reintroduce the neighbouring evidence the gate has just attenuated. This is consistent with the grid but not established by it: the one-factor-at-a-time experiment that would separate this mechanism from alternatives was not run. The practical consequence is that encoder and pooling head cannot be optimised independently, and since neither component's spread dominates the other's (Section 5.2), the combination has to be searched rather than composed.

6.4 Where the approach failed

Dual-stream pooling: a safety property that training never exercises. This is the clearest negative result. Property 3.6 states that dual-stream pooling cannot be worse than the conjunctive baseline, because its blend λ can reach 0 and recover it exactly. In practice it is the weakest head tested by a wide margin: mean WebTraffic accuracy

0.744 over its four encoder pairings against 0.90 and above for class-wise, softmax conjunctive and Top-k pooling, with one pairing collapsing to 0.556 (Table 8).

The property is about representational capacity, while the failure is one of optimisation. No term in Equation (33) pushes λ towards 0, so if the critical-instance stream of Equation (31) learns a poor query early in training, gradient descent has no incentive to return to the safe mean. The model *can* represent the baseline; it never finds it.

Per-class gated attention: no fallback at all. Gated attention is a weaker version of the same problem: it is the only proposed head with no blend parameter, so no setting of its weights reduces it to conjunctive pooling (Section 3.4.6). Its mean WebTraffic accuracy of 0.918 is respectable, but it is never the best partner for any encoder, unlike Top-k and class-wise pooling which each produce a top-ten model. Without a known-good configuration to fall back on, a poorly trained gate has nothing to recover to, and the head settles in the middle of the ranking rather than either winning or failing outright.

What both cases indicate. A property that holds by construction is only useful if training actually puts the model in the state where it applies. Neither head got there under the single configuration used across the whole study (Section 4.2). Two fixes follow directly: start λ near 0 so the model has to move away from the safe branch on purpose, or add a penalty that holds a blended head close to that branch early in training.

6.5 How the numbers should be read

AOPCR compares rows of one table, and nothing else. AOPCR is an area under a curve of raw confidence differences, so its scale follows the scale of the model's logits and has no fixed range (Section 2.4). Every model in Table 4 was trained under one configuration, so its AOPCR column is internally comparable; an AOPCR produced under a different configuration is not on the same scale and must not be placed beside it. Section B.4 measures how large that effect is on one architecture, and it is large.

Differences smaller than the seed spread are not results. Where three seeds are available the spread is not negligible: accuracy standard deviation ranges from 0.003 for SEA-Net gated + Top-k to 0.030 for SEA-Net input-gated + adaptive class-wise, and AOPCR is proportionally noisier still – 0.437 for SEA-Net input-gated + adaptive class-wise, about a sixth of its own mean, and 0.884 for the re-trained MILLET baseline (Table 4). This was applied as a rule in both directions. SEA-Net input-gated + adaptive class-wise's AOPCR advantage over SEA-Net bottleneck + Top-k (2.651 against 2.621) is far smaller than either model's standard deviation and is reported as noise, not as a result; equally, SEA-Net gated + Top-k's accuracy lead over SEA-Net bottleneck + Top-k (0.955 against 0.947) cannot be called decisive, because the second model's spread is 0.016. For the single-seed models no such test is possible, and those numbers are indicative rather than exact.

The training budget was the same for everyone, and it was short. The configuration of Table 3 trains for at most 400 epochs with patience 60. That was the budget 72 variants over 129 datasets allowed (Section A.5), and it applies equally to the baselines and to the SEA-Net models, so it does not favour either. It does mean the absolute accuracies here are lower than what a longer schedule would produce, on both sides. Section B.4 quantifies that on one architecture, and re-training the SEA-Net encoders under a longer schedule is the first item of future work.

6.6 Limitations

Six limitations bound what the results above support. **Explanation correctness is measured on one dataset**: NDCG@n needs per-time-step ground truth, which only WebTraffic provides, and WebTraffic is synthetic, so its important regions are generated rather than observed and it is not known how the result holds on real, noisier data. **The headline models were selected on that same dataset**, and Section 5.2 shows what that costs: over the 85 UCR datasets the re-trained baseline is ahead of both. **Most models have a single seed** – four have 3 seeds on WebTraffic and two of those also over the UCR archive, while the other 68 have one; where repeats exist the spread was large enough to change how a claim is phrased (Section 6.5), which is a reason for caution about the single-seed numbers rather than a reason to trust them. **Only univariate series were tested**: the encoder is indifferent to the number of input channels, so multivariate input should work mechanically, but that was not verified. **Faithfulness is not usefulness**, because AOPCR and NDCG@n both measure whether the stated explanation matches what the model used, and neither establishes that a person would find the highlighted time steps informative – that needs a human study. Finally, **nothing has run on a real microcontroller**: the cost measurements indicate that it should fit (Table 7), but that is not the same as a working deployment (objective O6, Table 6).

7 CONCLUSION

This internship addressed a question that arises whenever a time series classifier has to run on a constrained device: can such a model be made smaller *and* more interpretable at the same time, rather than trading one against the other? The starting point was MILLET (Early et al., 2024), which produces a faithful built-in explanation but needs about 424 K parameters.

The proposed solution is SEA-Net, an architecture in two interchangeable parts: an encoder built from multi-scale depthwise-separable dilated convolutional blocks that never collapse the time axis, in five variants plus two wrappers (Section 3.1), and seven MIL pooling heads (Section 3.4), each addressing a specific limitation of MILLET's conjunctive pooling and five of them provably reducing to it. Seventy-two combinations were evaluated on WebTraffic and the UCR archive against FCN, ResNet and MILLET baselines re-trained under an identical training configuration.

The main finding is that the three requirements of Section 1.1 are compatible. On WebTraffic, SEA-Net bottleneck + Top-k exceeds the re-trained MILLET baseline on accuracy, AOPCR and NDCG@n simultaneously while using 90 % fewer parameters (41,324 against 423,707), a third of the file size and less time per prediction; SEA-Net gated + Top-k is the most accurate model of the study at 0.955 with 61,740 parameters and the smallest seed-to-seed spread measured. Two results explain *why*: the parameter reduction targets channel mixing rather than temporal filtering, and the explanation gain comes from Top-k selection making the interpretation exactly zero where the model used no evidence – both supported by controlled ablations (Section 6). A third finding is that the encoder and the pooling head interact, so the best model is not obtained by combining the two individually best components.

The principal limitation is equally clear and constrains the claim above. The headline models were selected on WebTraffic, the only dataset with per-time-step ground truth, and over the 85 UCR datasets the re-trained MILLET baseline remains ahead on average (Section 5.2). A choice made on one dataset should not be expected to be optimal on 85 heterogeneous ones, and it was not. The remaining limitations are listed in Section 6.6. The objectives set at the start of the internship and their completion status are summarised in Table 6: five of the six were reached, and the sixth – deployment on a physical microcontroller – was not attempted.

7.1 FUTURE WORK

- **Re-train SEA-Net under a longer training configuration.** On the baseline architecture, a four-times-longer schedule is worth about 0.016 of UCR accuracy (Section B.4). Giving the SEA-Net encoders the same budget would show whether the UCR gap of Section 5.2 is architectural or only a budget effect. This is the single most informative experiment left.
- **Adapt further MIL pooling methods from the histopathology literature.** The two heads transferred here (Section 3.4.6, Section 3.4.7) come from that literature, and two more recent methods address exactly the failure mode observed in Section 6.4. DTFD-MIL (Zhang et al., 2022) splits a bag into pseudo-bags, trains a first-tier model on them and distils the resulting instance features into a second tier; this increases the number of effective training bags, which is a plausible remedy for a head whose attention is unstable early in training. ReMix (Yang et al., 2022) reduces a bag to a small set of representative instance prototypes and augments it by mixing them, which addresses the dilution of Section 2.3 at the level of the bag rather than the aggregator. Neither has been implemented here; both are proposed as the next pooling heads to try, with the caveat that whole-slide images contain thousands of instances per bag while these series contain around a thousand time steps, so the transfer is not automatic.
- **Make the dilation cap depend on the series length.** Over all 128 UCR datasets, accuracy falls with series length (correlation -0.34; 0.857 on the short half of the archive against 0.755 on the long half), which is what a fixed cap of 16 predicts and an effect far larger than any seed-to-seed spread measured (Table 12). Adapting the cap is a small change with a clear hypothesis behind it.
- **Correct the two heads that failed for optimisation reasons.** Initialising the dual-stream blend λ near zero, or penalising it away from the safe branch early in training, is a direct test of the diagnosis in Section 6.4.
- **Evaluate on a real dataset with per-time-step annotations,** so that explanation correctness no longer rests on a generator's own record of what it modified.
- **Deploy on the device.** Quantise SEA-Net bottleneck + Top-k (Jacob et al., 2018) and measure accuracy, latency and memory on a physical microcontroller, which is what objective O6 requires and the natural continuation of this project.

ACKNOWLEDGEMENTS

I thank my supervisor, Tanmoy Mondal, for his guidance throughout this internship, and the Fox team for their support. I also thank the Grid'5000 testbed (Balouek et al., 2013), whose GPU nodes made the 72-model study in this report possible.

REFERENCES

Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 2623–2631, 2019.

Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018.

Daniel Balouek, Alexandra Carpen Amarie, Ghislain Charrier, Frédéric Desprez, Emmanuel Jeannot, Emmanuel Jeanvoine, Adrien Lèbre, David Margery, Nicolas Niclausse, Lucas Nussbaum, Olivier Richard, Christian Pérez, Flavien Quesnel, Cyril Rohr, and Luc Sarzyniec. Adding virtualization capabilities to the Grid'5000 testbed. In *Cloud Computing and Services Science*, volume 367 of *Communications in Computer and Information Science*, pp. 3–20, 2013.

Colby Banbury, Vijay Janapa Reddi, Peter Torelli, Jeremy Holleman, Nat Jeffries, Csaba Kiraly, Pietro Montino, David Kanter, Sebastian Ahmed, Danilo Pau, Urmish Thakker, Antonio Torrini, Peter Warden, Jay Cordaro, Giuseppe Di Guglielmo, Javier Duarte, Stephen Gibellini, Videet Parekh, Honson Tran, Nhan Tran, Niu Wenxu, and Xu Xuesong. MLPerf tiny benchmark. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2021.

François Chollet. Xception: Deep learning with depthwise separable convolutions. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1251–1258, 2017.

Hoang Anh Dau, Anthony Bagnall, Kaveh Kamgar, Chin-Chia Michael Yeh, Yan Zhu, Shaghayegh Gharghabi, Chotirat Ann Ratanamahatana, and Eamonn Keogh. The UCR time series archive. *IEEE/CAA Journal of Automatica Sinica*, 6(6):1293–1305, 2019.

Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006.

Thomas G. Dietterich, Richard H. Lathrop, and Tomás Lozano-Pérez. Solving the multiple instance problem with axis-parallel rectangles. *Artificial Intelligence*, 89(1-2):31–71, 1997.

Joseph Early, Gavin K. C. Cheung, Kurt Cutajar, Hanting Xie, Jasmina Kandola, and Niall Twomey. Inherently interpretable time series classification via multiple instance learning. In *International Conference on Learning Representations (ICLR)*, 2024.

Noriaki Hashimoto, Daisuke Fukushima, Ryoichi Koga, Yusuke Takagi, Kaho Ko, Kei Kohno, Masato Nakaguro, Shigeo Nakamura, Hidekata Hontani, and Ichiro Takeuchi. Multi-scale domain-adversarial multiple-instance CNN for cancer subtype classification with unannotated histopathological images. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3852–3861, 2020.

Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.

Maximilian Ilse, Jakub M. Tomczak, and Max Welling. Attention-based deep multiple instance learning. In *International Conference on Machine Learning (ICML)*, pp. 2127–2136, 2018.

Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning (ICML)*, pp. 448–456, 2015.

Hassan Ismail Fawaz, Germain Forestier, Jonathan Weber, Lhassane Idoumghar, and Pierre-Alain Muller. Deep learning for time series classification: A review. *Data Mining and Knowledge Discovery*, 33(4):917–963, 2019.

Hassan Ismail Fawaz, Benjamin Lucas, Germain Forestier, Charlotte Pelletier, Daniel F. Schmidt, Jonathan Weber, Geoffrey I. Webb, Lhassane Idoumghar, Pierre-Alain Muller, and François Petitjean. Inceptiontime: Finding AlexNet for time series classification. *Data Mining and Knowledge Discovery*, 34(6):1936–1962, 2020.

Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2704–2713, 2018.

Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

Bin Li, Yin Li, and Kevin W. Eliceiri. Dual-stream multiple instance learning network for whole slide image classification with self-supervised contrastive learning. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 14318–14328, 2021.

Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 4765–4774, 2017.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 8026–8037, 2019.

Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "why should I trust you?": Explaining the predictions of any classifier. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1135–1144, 2016.

Cynthia Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5):206–215, 2019.

Wojciech Samek, Alexander Binder, Grégoire Montavon, Sebastian Lapuschkin, and Klaus-Robert Müller. Evaluating the visualization of what a deep neural network has learned. *IEEE Transactions on Neural Networks and Learning Systems*, 28(11): 2660–2673, 2017.

Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *International Conference on Machine Learning (ICML)*, pp. 3319–3328, 2017.

Zhiguang Wang, Weizhong Yan, and Tim Oates. Time series classification from scratch with deep neural networks: A strong baseline. In *International Joint Conference on Neural Networks (IJCNN)*, pp. 1578–1585, 2017.

Yunshi Wen, Tengfei Ma, Tsui-Wei Weng, Lam M. Nguyen, and Anak Agung Julius. Abstracted shapes as tokens – a generalizable and interpretable model for time-series classification. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://openreview.net/forum?id=pwKkNSuuEs.

Yunshi Wen, Tengfei Ma, Ronny Luss, Debarun Bhattacharjya, Achille Fokoue, and Anak Agung Julius. Shedding light on time series classification using interpretability gated networks. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://openreview.net/forum?id=n34taxF0TC.

Jiawei Yang, Hanbo Chen, Yu Zhao, Fan Yang, Yao Zhang, Lei He, and Jianhua Yao. ReMix: A general and efficient framework for multiple instance learning based whole slide image classification. In *Medical Image Computing and Computer Assisted Intervention (MICCAI)*, volume 13432 of *Lecture Notes in Computer Science*, pp. 35–45, 2022.

Hongrun Zhang, Yanda Meng, Yitian Zhao, Yihong Qiao, Xiaoyun Yang, Sarah E. Coupland, and Yalin Zheng. DTFD-MIL: Double-tier feature distillation multiple instance learning for histopathology whole slide image classification. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 18802–18812, 2022.

A Internship Context and Organisation

A.1 Host institution, team and supervision

The internship took place at the IRCICA / CRIStAL laboratory, within the FOX research team, which works on time series analysis across several application domains and on the hardware that runs it, in particular microprocessors and microcontroller units. The subject was titled "Time series classification using machine learning for edge devices (microcontrollers)", and the work divided roughly into 70 % time series classification and 30 % edge-device considerations, which is the balance reflected in this report.

I met my supervisor at least once a week, with day-to-day communication over Discord, and all code was kept in a shared Git repository he had access to. I wrote the code alone, but every idea was discussed before implementation: we read recent papers together, I proposed an approach and returned with the measured results. That cycle produced the sequence of variants in Section 3.

A.2 Computing environment

Code was written and small tests run on a local Windows machine with a 6 GB RTX GPU. All training reported here ran on the Grid'5000 testbed (Balouek et al., 2013), using the Lille and Sophia sites with the OAR job scheduler. The two environments are not identical – only Grid'5000 produced the numbers in this report – and both are documented in full in the repository README (Section 4.2).

> **My contribution.** I wrote the job submission scripts for both sites, and set up a monitoring system based on tmux and a Telegram bot that sent the accuracy of each finished model to my phone, so a failed or stalled job could be detected without keeping a session open.

A.3 The mission and how it was defined

Starting point. The subject began with a reading list from my supervisor: the MILLET paper (Early et al., 2024) and its public code, InterpGN (Wen et al., 2025), VQShape (Wen et al., 2024), the MIL pooling literature (Ilse et al., 2018; Li et al., 2021) and the TCN paper (Bai et al., 2018). Reproducing MILLET was step zero, not a contribution. One observation from that phase shaped the whole methodology: re-running the published model did not reproduce the published numbers exactly, which led to re-training every baseline under one identical configuration (Section 4.2) and, eventually, to the controlled comparison in Section B.4 that closes objective O2.

> **Existing work (reused).** The MILLET code was reused without modification. The only change was to wrap it inside the pipeline of Section 4.2 so that their models and mine could be trained and evaluated under identical conditions.

Problem statement. The question the internship had to answer was whether a substantially smaller encoder can match the accuracy of the MILLET baseline while producing better explanations. This is not obvious in either direction: reducing depth or width normally costs accuracy, and sharper explanations normally cost accuracy as well, so improving on all three axes at once is not the default outcome.

A.4 Objectives

Six objectives were agreed with my supervisor at the start, each with a criterion that could be measured rather than judged. Table 6 states them and the outcome of each; the conclusion (Section 7) refers to this table rather than repeating it. Five were reached and the sixth, running the model on a physical microcontroller, was not attempted.

Table 6: The six objectives, their measurable success criteria and the outcome. The evidence column gives the place in this report where each outcome can be checked.

Objective	Success criterion	Outcome	Evidence
O1. Reproduce the baselines	Compare the candidate architectures and select one as the basis for interpretability	Reached	MILLET was selected; FCN, ResNet and InceptionTime were all re-trained here under one identical configuration rather than quoted (Section 4.2).
O2. Match the published MILLET results	Re-run the MILLET model and compare against the published numbers	Reached	Under their own hyperparameters the re-implementation reaches 0.8434 over UCR against a published 0.8445, so the code reproduces; the rest is training budget (Table 13).
O3. Design a smaller encoder	$\geq$ 50 % fewer parameters at equal accuracy	Exceeded	41,324 against 423,707 parameters at higher accuracy on WebTraffic, a 90 % reduction (Table 4).
O4. Improve explanation quality	Higher AOPCR and NDCG@n on WebTraffic	Reached on WebTraffic	Both metrics improve over the re-trained baseline, and the κ ablation shows the gain comes from the intended mechanism (Section 6.2). Over the UCR archive the proposed heads do not lead, so this is not a general claim (Section 5.2).
O5. Build a reproducible benchmark	Any model reproducible from one command and one configuration file	Reached	Any encoder and pooling head pair trains from a single YAML file, and every figure and table here is regenerated by one command (Section 4.2).
O6. Test on a physical device	Measure the final model's performance on a microcontroller	Not attempted	The cost measurements indicate it should fit (41 K parameters, 1.43 MB, Table 7), comparable to the MLPerf Tiny reference model (Banbury et al., 2021), but no on-device test was run.

A.5 Constraints

Four constraints shaped the study and explain several choices made in Section 4. The first was **time**: four months in total, covering the literature review, the implementation, the experiments and this report. The second was **compute**: shared GPU nodes with a wall-clock limit per job and a queue, where a complete run over the 128 UCR datasets takes about five hours for one model and a job cannot always start when it is submitted – this is the direct reason most of the 72 variants have a single seed. The third was **fairness of the comparison**: every model had to be trained under one identical configuration, which meant no model could be tuned individually, so the numbers reported are not the best each model could reach. The fourth was **the evaluation data**: NDCG@n requires per-time-step labels, which only WebTraffic provides, so screening on WebTraffic was not a preference but a constraint, and Section 5.2 shows what it cost.

A.6 How the plan evolved

The initial plan was to reproduce MILLET and then shrink its encoder; two reading decisions changed it. VQShape (Wen et al., 2024) explains predictions through learned shapelets, which we judged more complex than the target required, so the MIL route was kept. Conversely, the histopathology MIL literature proved directly relevant: the multi-scale idea of MS-DA-MIL (Hashimoto et al., 2020) became the pyramid wrapper (Section 3.3.7) and the two-stream idea of DSMIL (Li et al., 2021) a pooling head (Section 3.4.7). Both transfers underperformed, and the results explain why: a whole-slide image is a bag of thousands of patches whose attention relies on a strong pre-trained feature extractor, whereas these series contain around 1,000 to 1,500 time steps and are encoded from scratch. That redirected the effort towards the TCN-based encoders and the heads derived directly from the three limitations of conjunctive pooling (Section 2.3). The final change was methodological: after the WebTraffic results looked clean, the full UCR experiments showed the picture was incomplete (Section 5.2), which led to adding the multi-seed runs and the controlled configuration comparison rather than reporting the first result alone.

A.7 Skills acquired

Scientific. Reproducing MILLET required going beyond the equations in the paper into the authors' code, then identifying why an honest re-implementation still missed the published number – the training configuration rather than the architecture. The same care shaped the protocol: re-training FCN, ResNet and InceptionTime under one configuration is what allows any difference in Section 5 to be attributed to the architecture. I also learned to test a metric before trusting it, after finding that the same model scores AOPCR 2.57 under one training configuration and 13.27 under another (Section B.4).

Technical. Designing every encoder and head around one fixed tensor contract is the only reason 72 variants could be trained without writing 72 model classes. Running them meant working on a shared HPC platform, reserving GPU nodes on Grid'5000 with OAR across two sites so that a queue at one did not stall the study. Making a single command rebuild every figure and table from the stored results repeatedly caught numbers that had silently gone out of date.

Professional. The most difficult update to prepare was bringing my supervisor a discrepancy I had not yet solved (objective O2) instead of debugging privately until it matched; that turned a private problem into a shared decision and produced a better experiment than I would have designed alone. The lesson I would keep is the last one: before the 85-dataset UCR experiments the WebTraffic numbers told a clean story, and running the same models over 85 further datasets showed that the story does not fully hold. Neither result is wrong; only the second is complete. That is the concrete form of not trusting a result verified once, and it is why Section 6.6 is a substantive part of this report.

B Supporting Results

The tables below are the ones the arguments of Section 5 and Section 6 rest on. The remaining material – the complete 72-model leaderboard, the encoder × pooling head grid, the per-dataset UCR results, the ensemble and Optuna experiments and the additional figures – is published with the code at `https://github.com/ubaidur404786/sea-net` (branch `main`).

B.1 Model cost

Section 5.1 states the headline of this table; the measurements behind it are given here.

Table 7: Cost of each model, measured on the real WebTraffic input (series of length 1,008, 10 classes) on one GPU. "One prediction" is the time to classify a single series. "Training time" is the wall-clock time to convergence including early stopping, over the 3 WebTraffic seeds.

Model	Params	Size (MB)	1 prediction (ms)	Train (s)
SEA-Net bottleneck + Top-k	**41,324**	**1.43**	0.131	117.3 ± 14.0
SEA-Net input-gated + adaptive class-wise	58,102	1.49	0.128	105.8 ± 37.2
SEA-Net gated + Top-k	61,740	1.50	0.128	91.3 ± 25.9
MILLET, re-trained here	423,707	4.11	0.203	50.4 ± 9.5
ResNet + conjunctive	506,331	4.42	0.139	38.1
FCN + conjunctive	267,035	3.47	**0.068**	20.8

The last column is the counterweight: the SEA-Net models take two to three times longer to train than the baseline, on every seed. Training time here is wall-clock time until early stopping fires, so it measures how many epochs a model needed rather than the cost of one epoch, and the baseline stops improving sooner. Prediction time is the fair per-step comparison, and there SEA-Net is faster.

B.2 Ablation studies

A. Effect of the pooling head, averaged over encoders. Averaged over every encoder it was paired with, the pooling head alone moves WebTraffic accuracy by almost 18 points.

Table 8: WebTraffic accuracy, AOPCR and NDCG@n averaged over every encoder each pooling head was paired with; n is the number of pairings averaged. The three MILLET heads at the bottom are shown for reference, but two of them were only ever paired with InceptionTime, so their $n = 1$ rows are not comparable with a head averaged over ten pairings.

Pooling head	Mean accuracy	Mean AOPCR	Mean NDCG@n	n
Class-wise conjunctive (Section 3.4.1)	0.921	2.322	0.692	10
Per-class gated attention (Section 3.4.6)	0.918	1.869	0.733	4
Adaptive class-wise (Section 3.4.3)	0.912	2.210	0.732	9
Softmax conjunctive (Section 3.4.2)	0.907	1.690	0.712	9
Top-k conjunctive (Section 3.4.4)	0.904	2.267	0.712	16
Attention-max (Section 3.4.5)	0.838	1.758	0.645	8
Dual-stream conjunctive (Section 3.4.7)	0.744	2.007	0.636	4
MILLET additive (Early et al., 2024)	0.942	1.579	0.729	1
MILLET attention (Early et al., 2024)	0.888	2.403	0.710	1
MILLET conjunctive (Early et al., 2024)	0.839	2.712	0.616	4

B. Effect of the encoder, averaged over pooling heads. Table 9 does the same on the encoder side. Its spread, 0.937 down to 0.771, is close in size to the pooling spread above, which is why Section 6.3 treats the two choices as comparably important.

Table 9: WebTraffic accuracy, AOPCR and NDCG@n averaged over every pooling head each encoder was paired with; † marks a wrapper rather than an encoder (Section 3.3). The bottom three rows are the baseline encoders, each paired only with conjunctive pooling.

Encoder	Mean accuracy	Mean AOPCR	Mean NDCG@n	n
SEA-Net multi-scale channels† (Section 3.3.6)	0.937	2.385	0.747	4
SEA-Net reconstruction-residual (Section 3.3.5)	0.924	2.315	0.719	5
SEA-Net input-gated (Section 3.3.3)	0.904	2.345	0.718	5
SEA-Net gated (Section 3.3.2)	0.902	2.056	0.710	19
SEA-Net base (Section 3.2)	0.898	1.933	0.694	12
SEA-Net bottleneck (Section 3.3.1)	0.884	2.209	0.694	8
SEA-Net spike/trend (Section 3.3.4)	0.858	1.814	0.677	7
SEA-Net multi-scale pyramid† (Section 3.3.7)	0.771	1.394	0.604	3
InceptionTime (MILLET) **(reused)**	0.887	2.569	0.677	1
ResNet **(reused)**	0.772	2.952	0.554	1
FCN **(reused)**	0.742	3.826	0.533	1

Two cautions apply to both tables. Several groups contain very few models and not every pair was trained, so the full encoder $\times$ pooling head grid published with the code is the more reliable view; and the bottleneck encoder's mean (0.884) is lower than its best pairing suggests, because it was also paired with the weak heads.

C. Effect of the Top-k fraction κ. This ablation tests Property 3.4 directly: the same model trained five times with only κ changed. All five runs use seed 0, so the $\kappa = 0.1$ row is SEA-Net bottleneck + Top-k's seed-0 result (0.938) rather than its 3-seed mean (0.947).

Table 10: Top-k fraction ablation on WebTraffic (seed $= 0$). The model is SEA-Net bottleneck + Top-k and only κ changes; $\kappa = 1.0$ is the setting that recovers class-wise conjunctive pooling exactly (Property 3.4).

κ	Accuracy	Loss	AOPCR	NDCG@n
0.05	0.888	0.401	2.288	0.715
0.10 *(default)*	0.938	0.299	2.778	**0.777**
0.25	**0.940**	**0.277**	2.648	0.733
0.50	0.882	0.418	**3.114**	0.732
1.00 *(= class-wise)*	0.908	0.395	2.436	0.722

Selecting a subset genuinely helps: $\kappa = 1.0$, exactly class-wise conjunctive pooling, scores 0.908, clearly below the 0.938–0.940 obtained at $\kappa = 0.1$ and 0.25, so the advantage of Top-k comes from the selection itself rather than from the encoder. These numbers are interpreted in Section 6.2.

D. Effect of the attention entropy term. Every model in the main study used $\lambda_{\text{focus}} = 0.01$ (Equation (33)), so a matched pair was trained with the term on and off, everything else identical, seed 0 in both cases.

Table 11: The attention entropy term of Equation (33), on and off, on WebTraffic (seed $= 0$). The model is SEA-Net bottleneck + Top-k and only λ_{focus} changes.

λ_{focus}	Accuracy	Loss	AOPCR	NDCG@n
0.01 *(on, the default)*	0.938	0.299	**2.778**	**0.777**
0.00 *(off)*	**0.950**	**0.263**	2.303	0.765

Switching the term off improves classification (0.950 against 0.938, at a lower loss) and degrades both explanation metrics, so keeping it on is the appropriate default for this work. This is a single matched pair on one seed, so it indicates a direction rather than a measured effect size.

B.3 Per-seed results

Table 12: Individual seed results (WebTraffic accuracy) for the four models trained with 3 seeds.

Model	Seed 0	Seed 1	Seed 2	Mean $\pm$ std
SEA-Net gated + Top-k	0.954	0.958	0.952	0.955 ± 0.003
SEA-Net bottleneck + Top-k	0.938	0.938	0.966	0.947 ± 0.016
SEA-Net input-gated + adaptive class-wise	0.938	0.880	0.898	0.905 ± 0.030
MILLET, re-trained here	0.894	0.876	0.892	0.887 ± 0.010

The three SEA-Net models behave differently, which affects how far each result can be trusted. SEA-Net gated + Top-k is stable, six thousandths between its best and worst seed. SEA-Net bottleneck + Top-k is tight on two seeds and jumps on the third, so its mean is raised by one good run. SEA-Net input-gated + adaptive class-wise is genuinely dispersed – one seed reaches 0.938 and the other two sit near 0.89 – so its standard deviation reflects real instability rather than one failed run, which is the evidence behind Section 6.5 treating its AOPCR advantage as unreliable.

B.4 Relation to the published MILLET results

The main body of this report compares only models re-trained here, under the single configuration of Table 3 (Section 4.2). This subsection answers the separate question that choice raises: how do those re-trained numbers relate to the accuracies MILLET published, and was the difference caused by the re-implementation or by the training configuration?

Table 13 answers it on WebTraffic, where four measurements are available for one and the same architecture. Early et al. (2024) report 0.924 averaged over five seeds, and 0.940 for their five-model ensemble. Loading their released weights and evaluating them on this pipeline gives 0.922, which is within the spread of their own five seeds and confirms that the evaluation code agrees. Re-training the architecture from scratch under MILLET's published

hyperparameters – 1500 epochs, no weight decay, no label smoothing – gives 0.920, again inside that spread. Under the configuration of Table 3, which trains for at most 400 epochs, the same architecture reaches 0.887.

Table 13: The MILLET baseline (InceptionTime + conjunctive pooling) measured four ways on WebTraffic, and on the 85 UCR datasets where a published mean exists. Only the last row is used anywhere in the main body of this report.

How the number was obtained	WebTraffic acc.	UCR-85 mean acc.
Published (Early et al., 2024), mean of 5 seeds	0.924	0.8445
Published (Early et al., 2024), 5-model ensemble	0.940	–
Their released weights, evaluated on this pipeline	0.922	–
Re-trained here under *their* hyperparameters	0.920	0.8434
Re-trained here under Table 3	**0.887**	**0.8274**

Two conclusions follow, and they are the reason objective O2 in Table 6 is counted as reached. First, the re-implementation is correct: under their own hyperparameters it reproduces their UCR mean to within about one thousandth (0.8434 against 0.8445), with a balanced per-dataset record of 26 wins, 32 ties and 26 losses against the published numbers. Second, the remaining difference is therefore the training budget, not the architecture and not the code. The gap it accounts for is about 0.016 on the UCR mean and 0.033 on WebTraffic. That figure cannot simply be added to the SEA-Net models – a longer schedule does not have to help every architecture by the same amount – but it is the right order of magnitude to keep in mind when reading the UCR results of Section 5.2, and it is why re-training SEA-Net under a longer schedule is the first item of future work in Section 7.1.

The same comparison shows that AOPCR is unnormalised. The two re-trained rows of Table 13 are the same encoder, the same pooling head, the same metric code and the same dataset, differing only in the training configuration. Their AOPCR values are 2.57 and **13.27**. A factor of five, from the configuration alone, on a metric that is supposed to measure explanation quality. AOPCR is built from raw confidence differences, so its scale follows the scale of the model's logits, and this is what that looks like when it is measured rather than argued. It is the reason the AOPCR column of Table 4 is only ever read across its own rows, and never against an AOPCR printed in another paper – including MILLET's own published 4.55.

B.5 Full hyperparameters

Every model in this report is one YAML file under `configs/models/`. Table 14 adds the values that Table 3 does not list, in particular the head-specific initial values. The commands that reproduce every result, and the two software environments used, are documented in the repository README (Section 4.2).

Table 14: Hyperparameters shared by every model configuration, plus the two encoder sizes used. The wide encoder is `sv2/seanet.yaml`; every model in Section 5 uses the narrow one.

Group	Parameter	Value
Encoder, wide	width d / blocks	128 / 6
Encoder, narrow	width d / blocks	64 / 4
Shared by both	kernel sizes $\mathcal{K}$	$\{5, 11, 23\}$, summed per block
	dilation cap $\delta_{\max}$	16
	dropout	0.2
Pooling	attention width d_a / dropout	8 / 0.2
	positional encoding	on
	Top-k fraction κ (Section 3.4.4)	0.1
	initial τ_0 (Section 3.4.2)	1.0
	initial sharpness β_0 (Section 3.4.3)	0.3
	initial blend λ_0, attention-max / dual-stream	0.5 / 0.05

Code availability. The codebase is organised as one module per stage of the pipeline under `seanet/` – data loading, encoder and pooling implementations, training, and results and figure generation – driven by the configuration files under `configs/`. It is released under the Apache License 2.0 and, per its `NOTICE` file, builds on Amazon's `MILTimeSeriesClassification` codebase (Copyright Amazon.com, Inc. or its affiliates), the original MILLET implementation (Early et al., 2024), so any redistribution must preserve that attribution and its licence terms. The repository is not yet public; access can be granted on request: `https://github.com/ubaidur404786/sea-net`, branch `main`.